



多媒体技术

多媒体压缩技术

内容提纲

- 编码和压缩的基本概念
- 数据压缩的重要性
- 数据压缩方法分类
- 数据压缩基础

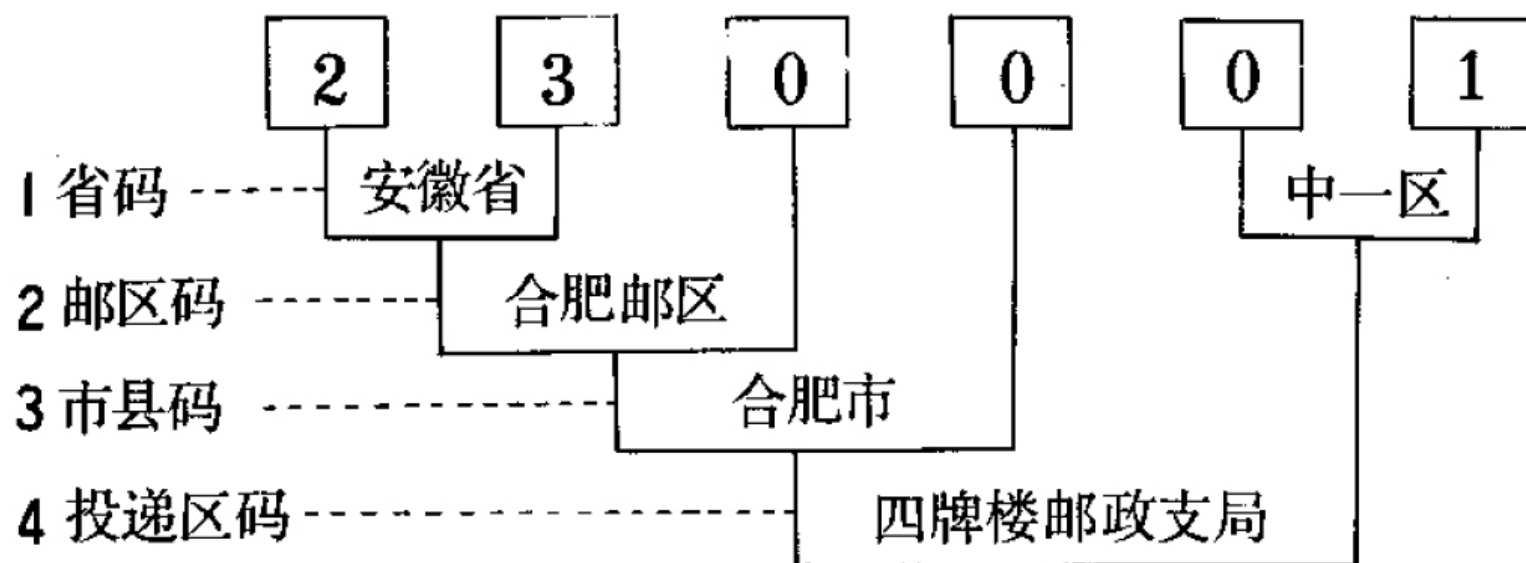
内容提纲

- 编码和压缩的基本概念
- 数据压缩的重要性
- 数据压缩方法分类
- 数据压缩基础

什么是编码



什么是编码



什么是编码

- 编码就是对应。例如：把字符转换为比特串。
- 对数字信息的编码表现为从一个比特流转换为另一个比特流。
- 这种对应（或形式上的转换）具有不同的用途：压缩、加密、提高传输过程中的抗干扰能力、作为唯一的标识等等。

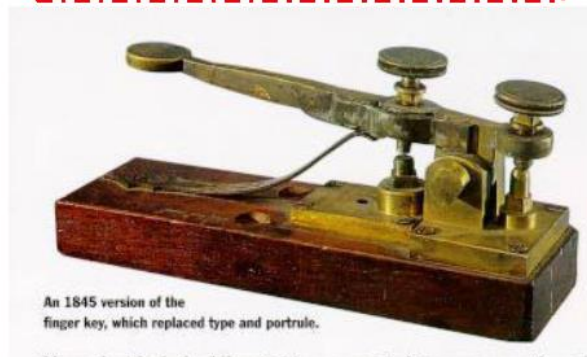
_ ↔ 00000
A ↔ 00001
B ↔ 00002
⋮ ... ⋮
Z ↔ 11010

, ↔ 11011
. ↔ 11100
? ↔ 11101
: ↔ 11110
; ↔ 11111

什么是编码

摩尔斯电码 (Morse code) 是一种时通时断的信号代码，通过不同的排列顺序来表达不同的英文字母、数字和标点符号。它发明于1837年，发明者是美国人塞缪尔·莫尔斯。摩尔斯电码是一种早期的数字化通信形式，它的代码包括五种：点、划、点和划之间的停顿、每个字符间短的停顿（在点和划之间）、每个词之间中等的停顿以及句子之间长的停顿。

Samuel Morse, telegraph
key circa 1860



A{—}滴答
B{—}答滴滴滴
C{—}答滴滴滴
D{—}答滴滴滴
E{—}滴
F{—}滴滴答滴
G{—}答答滴
H{—}滴滴滴滴
I{—}滴
J{—}滴滴答答
K{—}答滴滴滴
L{—}滴滴滴滴
M{—}答答
N{—}答滴
O{—}答答答
P{—}滴滴答滴
Q{—}滴滴答滴
R{—}滴滴滴
S{—}滴滴滴
T{—}答
U{—}滴滴答
V{—}滴滴滴滴
W{—}滴滴答
X{—}滴滴滴滴
Y{—}滴滴滴滴
Z{—}滴滴滴滴
1{—}———
2{—}———
3{—}———
4{—}———
5{—}———
6{—}———
7{—}———
8{—}———
9{—}———
0{—}———



Samuel Morse photo by
student Mathew Brady, 1845

<http://www.learnmorsecode.com/>

生活中的编码

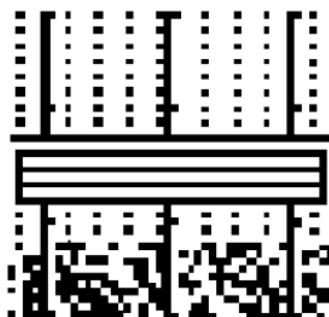
- 身份证、电话区号和邮政编码
- 汉字的编码：GB2312、BIG5
- 网卡号、手机设备号、IP地址
- 文件的后缀名称
- 商品的一维条码
- 用于存储信息的二维条码
- RFID（射频识别技术）
- Windows操作系统里的“全球识别编码”

二维条码实例

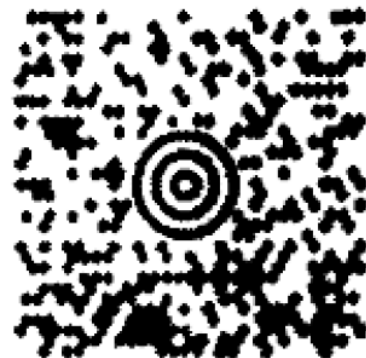
- 二维条码已研制出多种码制，常见的有PDF417，QR Code，Code 49，Code 16K，Code One等。



QR Code 条码



code-one



maxicode



code49



PDF417



Code 16K

Source: http://www.guardit.cn/2D/2d_zhishi_pic.htm

RFID (射频识别技术)



含RFID标签的运输工人身份识别



含RFID标签的上海世博会门票



含RFID标签的图书



含RFID标签的北京奥运会门票



含RFID标签的巴西世界杯门票



RFID在身份证中的应用

多媒体数据的编码

- 首先需要表示媒体信息
- 在最大程度上压缩比特流
- 适合于在网络上传输



什么是压缩

- 两种表现形式
 - 在等同的空间中容纳更多的信息
 - 在等同的时间内表现更多的信息
- 比特流的压缩
 - 用相同长度的比特流表示更多的信息
 - 用尽量短的比特流表达尽量多的信息
- 压缩的目的
 - 相同的代价处理更多的信息
 - 处理等量信息的代价减小
 - 提高媒体信息处理的效率
 - 提高媒体信息处理的能力

在哪里能看到压缩



寥寥数笔，勾勒出人物的动态

在哪里能看到压缩

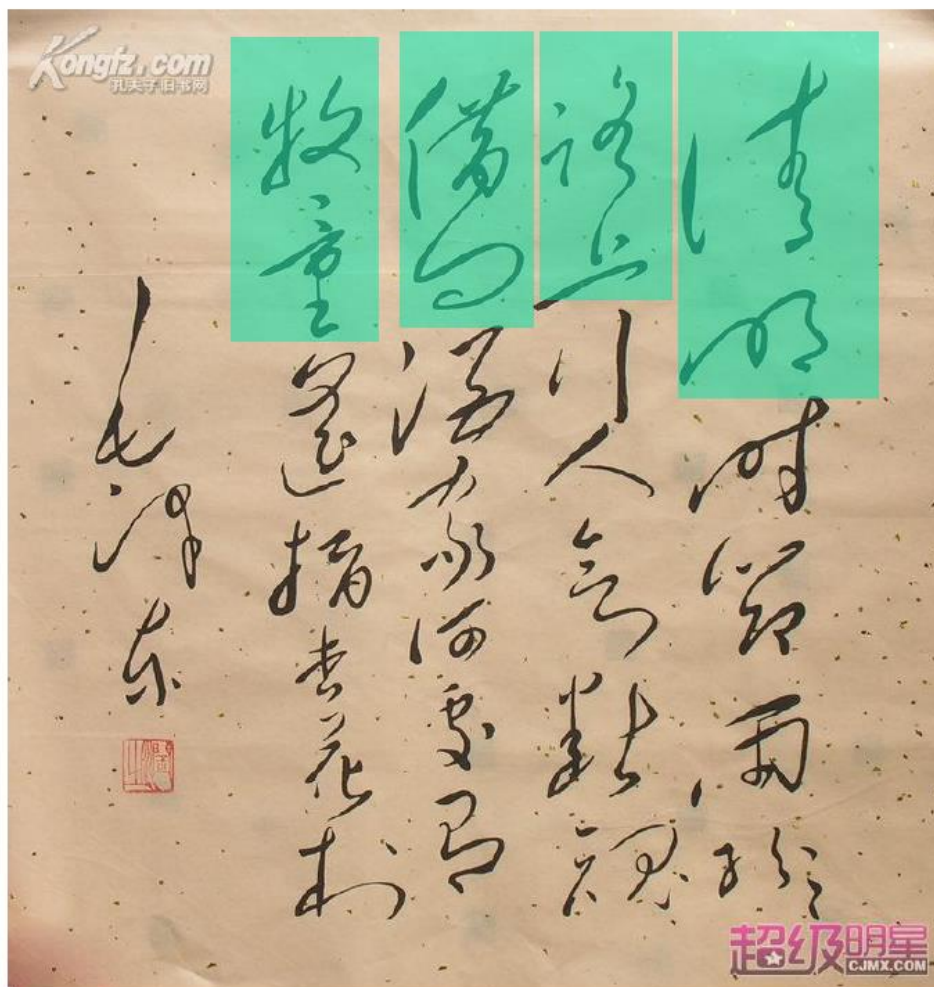


- 听的繁体字是听拆字就是“十目一心耳为王”也就是说要多听多看用心记，虚心听取别人的意见和合理的建议。

生活中的压缩例子

- 成语：字字珠玑
- 歇后语：高俅当太尉——一步登天
- 漫画
- 网络聊天术语：我伙呆，何弃疗，城会玩
- 英语缩略语：USA, IT, XMU
- 各种压缩软件 (winrar/7-zip/kuaizip)

如何进行压缩



每句前两个字
均可删去

编码和压缩的关系

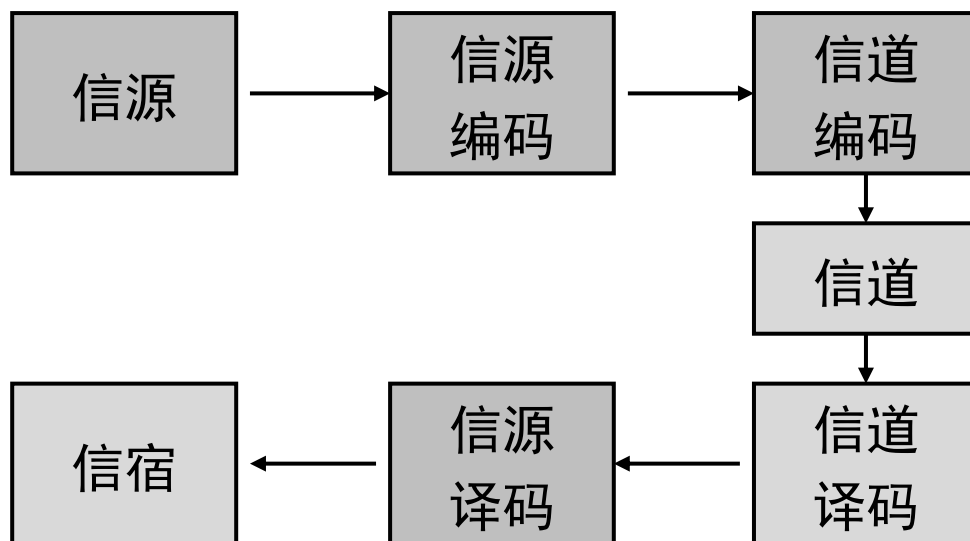
- 压缩是一种特殊的编码
- 压缩 = 模型+ 编码
- 数字压缩技术建立在数字编码基础之上

内容提纲

- 编码和压缩的基本概念
- 数据压缩的重要性
- 数据压缩方法分类
- 数据压缩基础

多媒体数据压缩的重要性

- 多媒体信息的数据海量性，与当前硬件技术所能提供的计算机存储资源和网络带宽之间有很大差距
- 目前，数字化的媒体信息数据以压缩形式存储和传输仍将是唯一选择



多媒体数据压缩的重要性

- 多媒体技术的处理对象
 - 三维图形
 - 立体声
 - 彩色全屏幕运动画面
- 数字化的视频和音频信号的数据量之大是非常惊人的

多媒体数据压缩的重要性

- 几个未经压缩的数字化信息的例子
 - 一页在B5 (约180mm×255mm) 纸上的文件, 若以中等分辨率 (300 dpi) 扫描仪进行数字化, 其数据量约6.61MB/页。一片650MB的CD-ROM, 可存98页
 - 双通道立体声激光唱盘 (CD-A) , 采样频率44.1KHz, 采样精度16位/样本, 一个650MB的CD-ROM, 可存约1小时的音乐

多媒体数据压缩的重要性

- PAL制的数字电视图像, 4:4:4采样格式

- 每帧数据量 $720 \times 576 \times 3 = 1.24 \text{ MB}$

- 每秒数据量 $1.24 \times 25 = 31.3 \text{ MB/s}$

- 一片CD-ROM可存帧数

$$650 / 1.24 = 0.524 \text{ 千帧/片}$$

- 一片CD-ROM可存节目时间

$$650 / 31.1 = 20.9 \text{ 秒/片}$$

多媒体数据压缩的重要性

- 遥感图像：陆地卫星（LandSat-3），其水平、垂直分辨率分别为2340和3240，四波段、采样精度7位

- 一幅图像的数据量为：

$$2340 \times 3240 \times 7 \times 4 = 212 \text{ Mb}$$

- 按每天30幅计算，每天的数据量为

$$212 \times 30 = 6.36 \text{ Gb}$$

- 每年的数据量高达 2300 Gb

多媒体数据压缩的重要性

- 庞大的数据量是多媒体技术发展中的一个非常棘手的瓶颈问题
- 通过数据压缩手段把信息数据量压下来，以压缩形式存储和传输
 - 紧缩节约了存储空间
 - 提高了通信干线的传输效率
 - 使计算机实时处理音频、视频信息，以保证播放出高质量的视频、音频节目成为可能

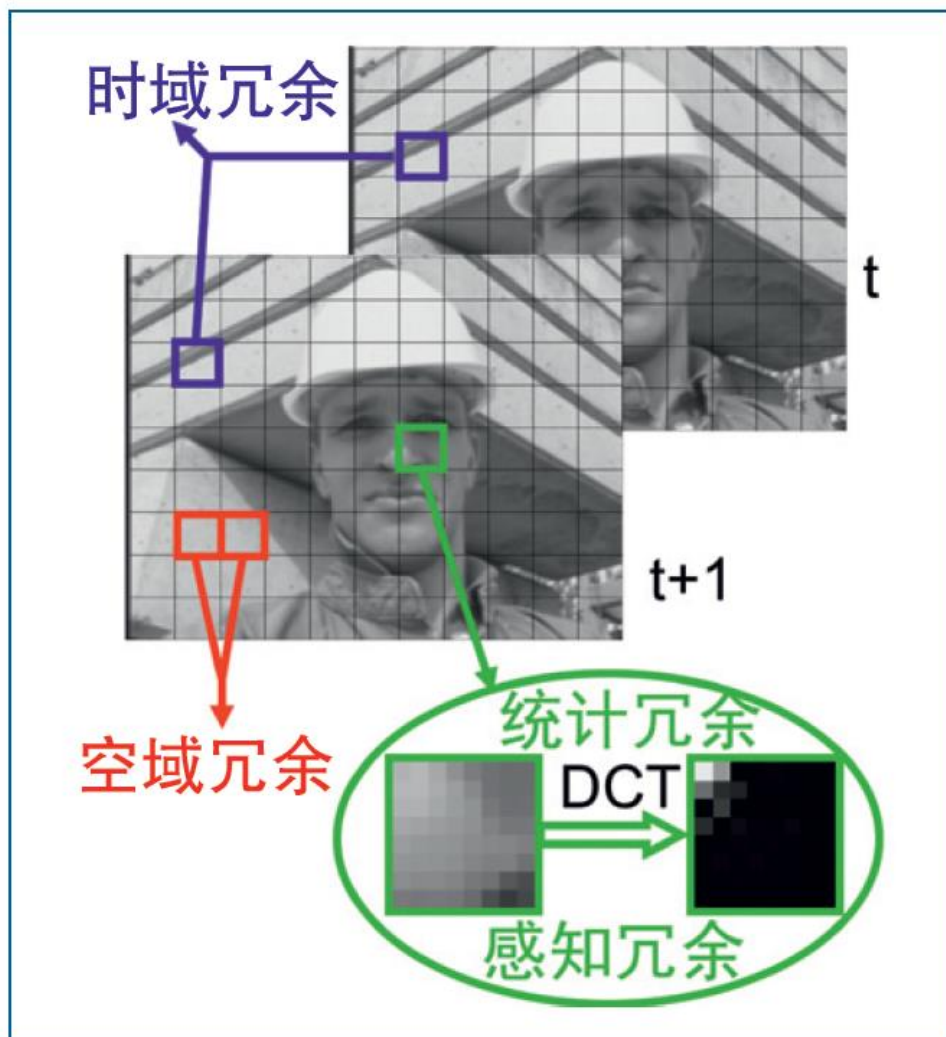
多媒体数据压缩的可能性

- 人们研究发现，图像数据表示中存在着大量的**冗余 (redundancy)**
 - 通过去除那些冗余数据可以使原始图像数据极大地减少。**图像数据压缩技术就是研究如何利用图像数据的冗余性来减少图像数据量的方法**
 - 随着对人类视觉系统和图像模型的进一步研究，人们可能会发现更多的冗余性，使图像数据压缩编码的可能性越来越大，从而推动图像压缩技术的进一步发展

多媒体数据压缩的可能性

空域冗余：图像在空间上存在很大相关性，如：相邻像素值重复或非常接近。

时域冗余：视频图像在时间上存在很大相关性。



多媒体数据压缩的可能性

- **视觉冗余（感知冗余）**：人眼感受不到图像中的一些复杂细节信息，丢弃这些信息不会影响视觉感受
 - 视觉系统对图像的亮度和色彩度的敏感性相差很大（YUV、YIQ颜色空间）
 - 人眼对高频信息不太敏感，对纹理丰富的内容区域进行较粗的量化，以提升编码效率

多媒体数据压缩的可能性



Lena.bmp (原图)



Lena.jpg (压缩率18.4)

多媒体数据压缩方法的应用

- 多媒体数据压缩算法及标准广泛应用于：
 - 多媒体计算机、多媒体数据库、常规的电视数字化、高清晰度电视(HDTV)以及交互式电视(ITV)系统中
- 目前正在开展的应用项目有：
 - 可视电话、视频会议、多媒体电子邮件、电子出版物、家庭卫星广播业务(BSS)、地面数字电视广播(DTTB)、电子影院(EC)、电子新闻采集系统(ENG)、个人通信(IPC)、网络数据库(NDB)、家庭电视剧场(HTT)、遥控监视(RVS)以及电视点播系统(VOD)等

内容提纲

- 编码和压缩的基本概念
- 数据压缩的重要性
- 数据压缩方法分类
- 数据压缩基础

多媒体数据压缩方法的分类

- 质量有无损失
 - 有损失编码、无损失编码
- 作用域
 - 空间方法、变换方法、混合方法
- 是否自适应
 - 自适应性编码、非自适应性编码

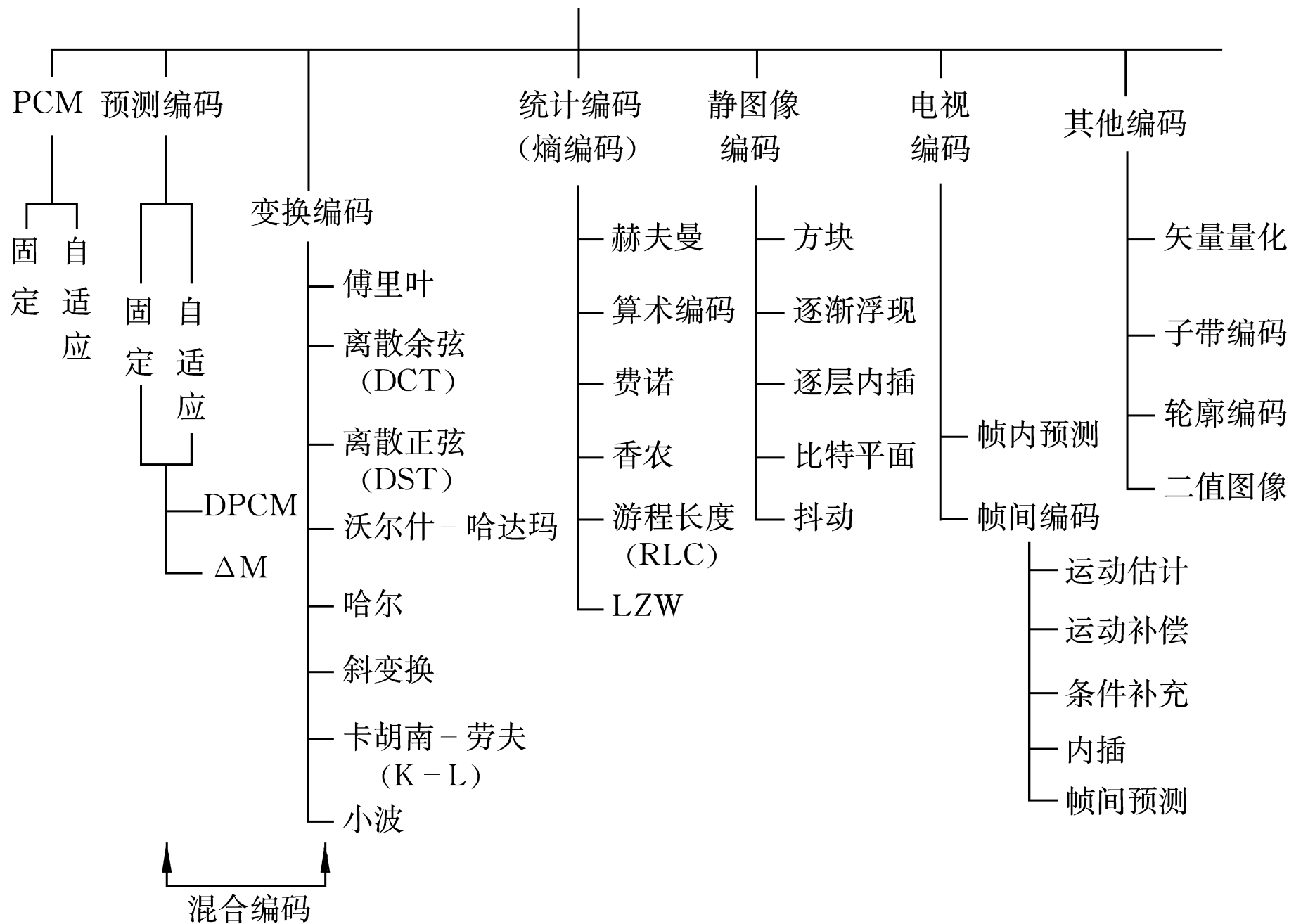
无损压缩（无损编码）

- 无损压缩是指使用压缩后的数据进行重构(或者叫做还原，解压缩)，重构后的数据与原来的数据完全相同；无损压缩用于要求重构的信号与原始信号完全一致的场合。一个很常见的例子是磁盘文件的压缩。根据目前的技术水平，无损压缩算法一般可以把普通文件的数据压缩到原来的 $1/2 \sim 1/4$ 。一些常用的无损压缩算法有霍夫曼 (Huffman) 算法和 LZW(Lenpel-Ziv & Welch)压缩算法。

有损压缩（有损编码）

- 有损压缩是指使用压缩后的数据进行重构，**重构后的数据与原来的数据有所不同，但不会让人对原始资料表达的信息造成误解。**有损压缩适用于重构信号不一定非要和原始信号完全相同的场合。例如，图像和声音的压缩就可以采用有损压缩，因为其中包含的数据往往多于我们的视觉系统和听觉系统所能接收的信息，丢掉一些数据而不至于对声音或者图像所表达的意思产生误解，但可大大提高压缩比。

多媒体数据编码算法



多媒体数据压缩方法的分类

- 脉冲编码调制 (PCM)
 - 它实际上是连续模拟信号的数字采样表示。PCM编码器和解码器位于一个图像编码系统的起点和终点, 它们实际上分别是A/D和D/A转换器
- 其它编码方法通常都是在多媒体数据模拟信号经过PCM编码后再进行的

多媒体数据压缩方法的分类

- 预测编码

- 编码器记录与传输的不是样本的真实值，而是它与预测值的差
- 由于空间的相关性，真实值和预测值的差值变化范围远远小于真实值的变化范围，因而可以采用较少的位数来表示
- 差分脉冲编码调制（DPCM）

多媒体数据压缩方法的分类

- 变换编码

- 主要思想：利用图像块内像素值之间的相关性，把图像变换到一组新的基上，使得能量集中到少数几个变换系数上，通过存储这些系数达到压缩的目的
- 由于对整幅图像进行变换的计算量太大，所以一般把原始图像分成许多个矩形区域子图像独立进行变换
- KLT、DCT、DFT 等

多媒体数据压缩方法的分类

- 统计编码

- 对于出现频率大的符号用较少的位数表示，而对于出现频率小的符号用较多的位数来表示
- 编码效率取决于信源的概率分布
- Huffman编码、算术编码、游程编码

- 混合编码

- 指合并变换和预测技术的编码方法
- 例如：对动态图像而言，空域的二维变换再加上时间方向上的DPCM预测

内容提纲

- 编码和压缩的基本概念
- 数据压缩的重要性
- 数据压缩方法分类
- **数据压缩基础**

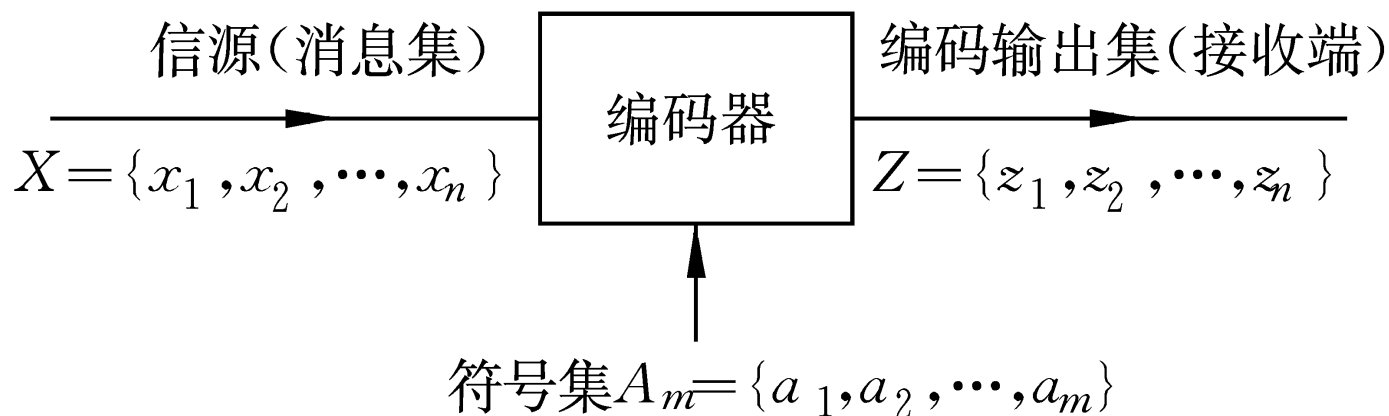
统计编码

统计编码

- **数据压缩的理论基础是信息论**。根据信息论原理，可以找到最佳数据压缩编码方法，数据压缩的理论极限是信息熵
- 无失真编码（熵编码）
 - 编码过程中不丢失信息，即要求保存信息熵
- 有失真编码
 - 可允许信息部分损失以换取较高的压缩比

信息量和信息熵

• 编码器模型图



- X 是消息集, 由几个信号单元 x_j 构成 ($j=1,2,\dots,n$)
- Z 是输出集, 由几个码子 z_j 构成 ($j=1,2,\dots,n$), z_j 与 x_j ——对应
- A_m 是符号集, 由 m 个码元 a_i 构成 ($i=1,2,\dots,m$), 码元组成输出码字
- 问题: 当信源发出某个随机事件(消息) x_j 后, 接收端收到一个相应的码子 z_j , 所收到的码子中包含多大的信息量?

信息量和信息熵

- 信息是用不确定性的度量定义的
 - 一个消息的可能性愈小，其信息愈多
 - 而消息的可能性愈大，则其信息愈少
- 信息论定义了一种度量信息量的方法

$$I(x_j) = -\log_a P(x_j) \quad j=1,2,\dots,n$$

- $P(x_j)$: 信源 X 发出 x_j 的先验概率
- $I(x_j)$: x_j 发生后的自信息量，其含义是信源 X 发出 x_j 这个消息（随机事件）后，接收端收到信息量的度量

信息量和信息熵

- **信源的熵 $H(X)$** 定义为：信源 X 发出的 n 个随机事件 x_j ($j=1,2,\dots,n$) 的自信息量的统计平均,

$$\begin{aligned} H(X) &= E\{I(x_j)\} = \sum_{j=1}^n P(x_j) I(x_j) \\ &= -\sum_{j=1}^n P(x_j) \log_a P(x_j) \end{aligned}$$

- 信源的熵 $H(X)$ ：信源 X 发出任意一个随机变量的平均信息量
- 当 a 取2时， $H(X)$ 的单位为“比特” (bit) 或者“香农” (Sh)

信息量和信息熵

- 信息熵提出者——香农



正在贝尔实验室中工作的香农, 1955.

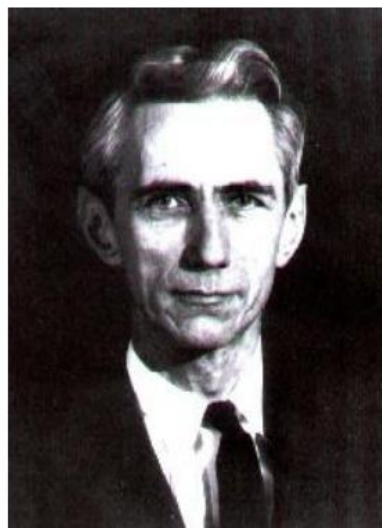


克劳德·艾尔伍德·香农 (Claude Elwood Shannon, 1916年4月30日—2001年2月26日) 是美国数学家、信息论的创始人。香农提出了信息熵的概念，为信息论和数字通信奠定了基础。



香农是使我们的世界能进行即时通信的少数科学家和思想家之一，他的一生主要有两大贡献：一是信息理论、信息熵的概念；另一是符号逻辑和开关理论。

Claude Shannon



Mrs. Shannon with the statue. (photograph courtesy of J. O'Sullivan)
<http://www.eecs.umich.edu/shannonstatue/>

信息量和信息熵

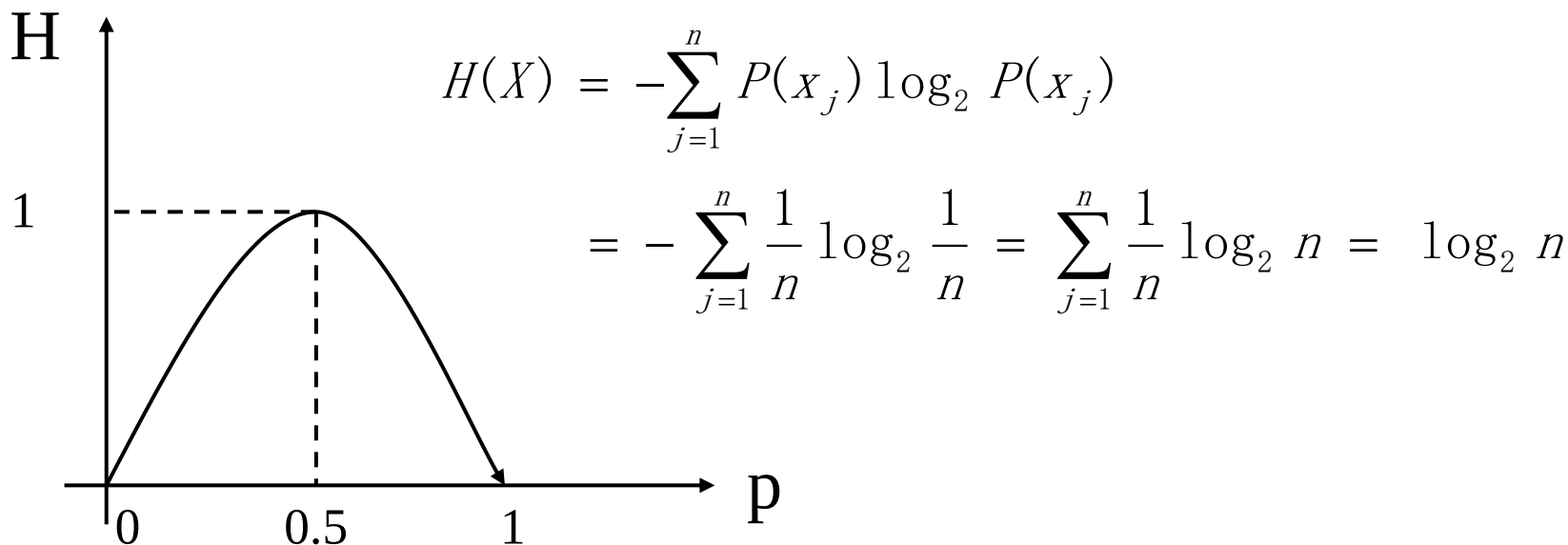
- 信息熵提出者——香农

- 香农 (Claude Elwood Shannon) 1916年生于美国，40年获得MIT数学博士学位和电子工程硕士学位。41年加入了贝尔实验室数学部，48年发表《通讯的数学原理》，该文奠定了信息论的基础。文中用非常简洁的数学公式定义了信息时代的基本概念：熵。在此基础上又定义了信道容量，指出了用降低传输速率来换取高保真通讯的可能性。这些贡献对通信工业具有革命性的影响。



信息量和信息熵

- 熵的大小与信源的概率分布有着密切联系
- 等概率事件的熵最大**：当信源中各事件为等概率分布时，熵具有极大值 $\log_2 n$ 。n为信源中事件个数



信息量和信息熵

- 以 $n=8$ 为例, 当 $p(x_1)=p(x_2)=\dots=p(x_8)=1/8$ 时,

$$H(X) = -\sum_{j=1}^8 \frac{1}{8} \log_2 \frac{1}{8} = 3 \text{ bits}$$

- 当 $p(x_1)=1$, 必然有: $p(x_2)=p(x_3)=\dots=p(x_8)=0$,

$$H(X) = -p(x_1) \log_2 p(x_1) = 0$$

- 熵的范围为: $0 \leq H(X) \leq \log_2 N$

信息量和信息熵

- 在编码中用熵衡量是否为最佳编码。令 $\tilde{N}$ 表示编码器输出码子的平均码长：

$$\tilde{N} = \sum_{j=1}^n p_j * N_j$$

- 熵值是平均码长的下限：**当分配给 x_j 的比特数 $N_j = I(x_j) = -\log_2 p(x_j)$ 时，平均码长取极小值，即达到压缩极限。
 - 当 $\tilde{N} \gg H(x)$ ，有冗余，不是最佳
 - 当 $\tilde{N} \approx H(x)$ ，最佳编码（ $\tilde{N}$ 稍大于 $H(x)$ ）
 - 当 $\tilde{N} < H(x)$ ，不可能

Huffman编码

- **最佳编码定理**

- 在变字长码中，对于出现概率大的消息符号编以短字长的码，对于出现概率小的消息符号编以长字长的码，如果码子长度严格按照符号概率的大小的相反顺序排列，则平均码子长度一定小于按任何其他符号顺序排列方式得到的码子长度

- 由David A. Huffman在MIT攻读博士期间发明

Huffman编码

霍夫曼编码的具体步骤如下：

- 1) 将信源符号的概率按减小的顺序排队。
- 2) 把两个最小的概率相加，并继续这一步骤，始终将较高的概率分支放在右边，直到最后变成概率 1 。
- 3) 画出由概率 1 处到每个信源符号的路径，顺序记下沿路径的 0 和 1 ，所得就是该符号的霍夫曼码字。
- 4) 将每对组合的左边一个指定为0，右边一个指定为1（或相反）。

Huffman编码

- 例子：有一幅16个像素组成的灰度图像，灰度共有5级分别用符号A、B、C、D和E表示，如下表所示：

灰度	A	B	C	D	E
个数	2	8	2	3	1

- 如果用3位表示5个等级的灰度值，也就是每个像素用3位表示，编码图像总共需要48比特

Huffman编码

像素	A	B	C	D	E
个数	2	8	2	3	1
频度	2/16	8/16	2/16	3/16	1/16
信息量	3	1	3	1.585	4

- 计算每个像素的信息量：

$$I(x_j) = -\log_2 P(x_j) \quad j=1,2,\dots,5$$

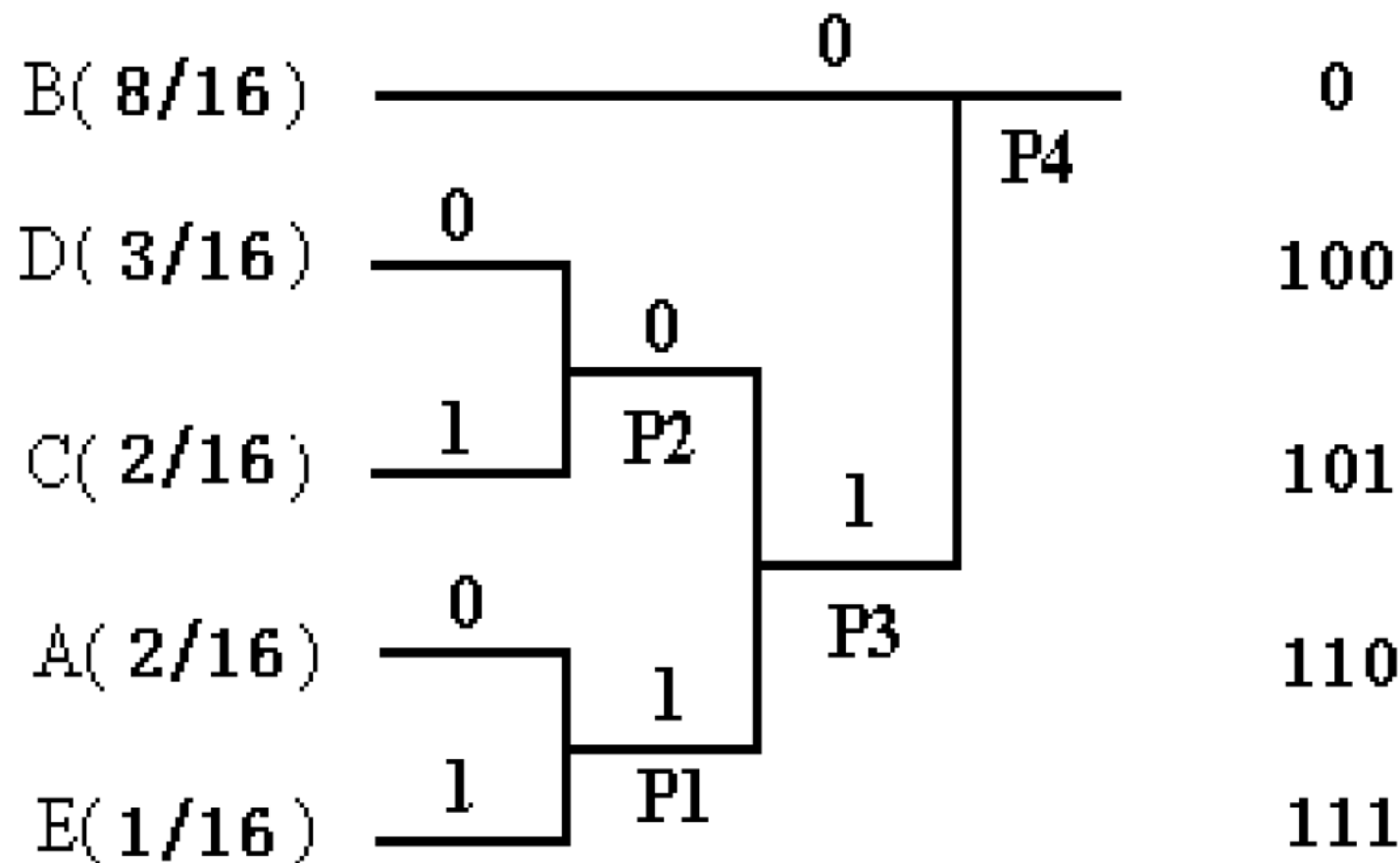
- 所有像素的熵：
$$H(X) = 2/16 * 3 + 8/16 * 1 + 2/16 * 3 + 3/16 * 1.585 + 1/16 * 4 = 1.6 \text{ bit}$$

Huffman编码

像素	A	B	C	D	E
个数	2	8	2	3	1
频度	2/16	8/16	2/16	3/16	1/16
信息量	3	1	3	1.585	4

- 熵是1.6，表示每个像素不需要3位代码表示，用1.6位就可以，16个像素共需要25.6位。
- 理论上能达到压缩比： $1.6 / 3 = 0.533$

Huffman编码过程



Huffman编码过程



像素	个数	频度	码	码长
A	2	1/8	110	3bit
B	8	1/2	0	1bit
C	2	1/8	101	3bit
D	3	3/16	100	3bit
E	1	1/16	111	3bit

- 平均码长: $\tilde{N} = 1/8 * 3 + 1/2 * 1 + 1/8 * 3 + 3/16 * 3 + 1/16 * 3$
 $= 2 \text{ bit}$

Huffman编码

- **总结如下特点：**

- 平均码长 $\tilde{N} > H$ (熵)
- 平均码长 $\tilde{N} < 3$ bits (等长码需要的比特数)
- 保证解码的唯一性，短码字不构成长码字的前缀
- 在接收端需保存一个与发送端相同的Huffman码表 (输入与Huffman的对应表)

Huffman编码

- **Huffman编码的局限**

- 如果信源事件的概率分布不是 2^{-n} 次方形式，编码是否可以达到最佳？原因...
- 假设某个字符的出现概率为80%，该字符事实上只需要 $-\log_2(0.8) = 0.322$ 位编码，但 Huffman 编码一定会为其分配一位 0 或一位 1 的编码
 - 可以想象，整个信息的 80% 在压缩后几乎相当于理想长度的 3 倍左右，压缩效果可想而知。

算术编码

- 基本思想：算术编码不是将单个信源符号映射成一个码字，而是把整个信源表示为实数线上的0到1之间的一个区间，其长度等于该序列的概率，再在该区间内选择一个代表性的小数，转化为二进制作作为实际的编码输出
 - 消息序列中的每个元素都要用来缩短这个区间。消息序列中元素越多，所得到的区间就越小，表示这个区间所需的位数就更多
 - 采用算术编码每个符号的平均编码长度可以为小数，因此可以更加接近无损压缩的熵极限

算术编码

- **新子区间的起始位置** = 前子区间的起始位置 + 当前符号的区间左端 $\times$ 前子区间长度
- **新子区间的长度** = 前子区间的长度 $\times$ 当前符号的概率（等价于范围长度）
- 最后得到的子区间的长度决定了表示该区域内的某一个数所需的位数。

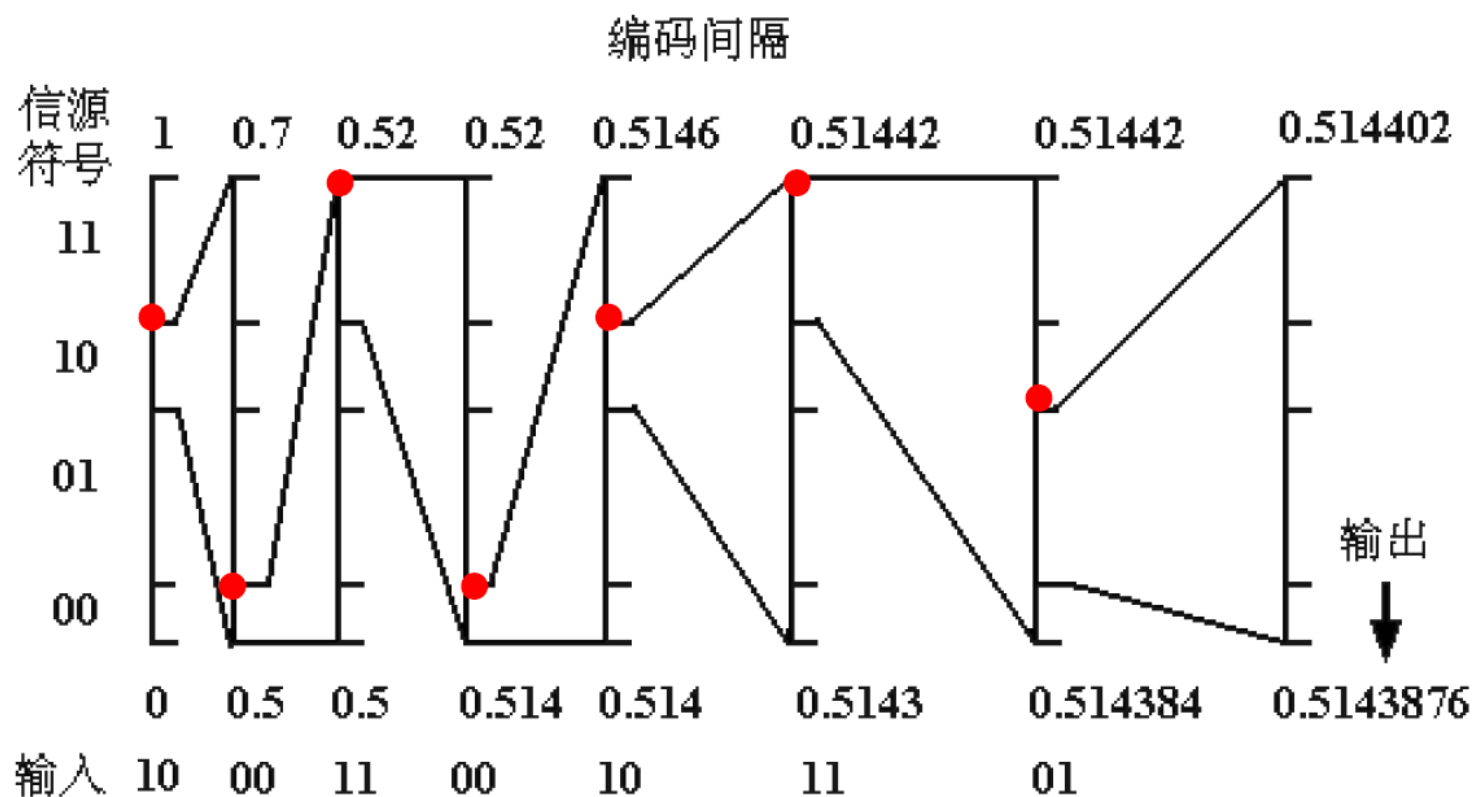
算术编码举例1

- 假设信源符号为{00,01,10,11}，这些符号的概率分别为{0.1,0.4,0.2,0.3}，根据这些概率可把间隔[0,1)分成4个子间隔：

[0,0.1),[0.1,0.5),[0.5,0.7),[0.7,1)，二进制消息序列的输入为：10 00 11 00 10 11 01

符号	00	01	10	11
概率	0.1	0.4	0.2	0.3
初始编码间隔	[0, 0.1)	[0.1, 0.5)	[0.5, 0.7)	[0.7, 1)

算术编码举例1



算术编码伪代码

Set low to 0.0

Set high to 1.0

While there are still input symbols do

 get an input symbol

$\text{code_range} = \text{high} - \text{low}$.

$\text{high} = \text{low} + \text{range} * \text{high_range}(\text{symbol})$

$\text{low} = \text{low} + \text{range} * \text{low_range}(\text{symbol})$

End of While

output [low, high)

算术编码举例2

字符	a	e	i	o	u
概率	0.2	0.3	0.1	0.2	0.2

范围 [0,0.2), [0.2,0.5), [0.5,0.6), [0.6,0.8), [0.8,1)

如果编码数据串为：eai。编码取[0, 1) 之间实数，

令： $low=0, high=1, range=high-low$, $range_{low}$ 为编码字符分配的低端， $range_{high}$ 为编码字符分配的高端，下一个字符新的 $low, high$ 按下式计算：

$$low=low+range*range_{low}$$

$$high=low+range*range_{high}$$

算术编码举例2

对字符e: $\text{range} = \text{high} - \text{low} = 1 - 0 = 1$

$\text{low} = 0 + 1 * 0.2 = 0.2$, $\text{high} = 0 + 1 * 0.5 = 0.5$

对字符a: $\text{range} = \text{high} - \text{low} = 0.5 - 0.2 = 0.3$

$\text{low} = 0.2 + 0.3 * 0 = 0.2$, $\text{high} = 0.2 + 0.3 * 0.2 = 0.26$

对字符i: $\text{range} = \text{high} - \text{low} = 0.26 - 0.2 = 0.06$

$\text{low} = 0.2 + 0.06 * 0.5 = 0.23$,

$\text{high} = 0.2 + 0.06 * 0.6 = 0.236$

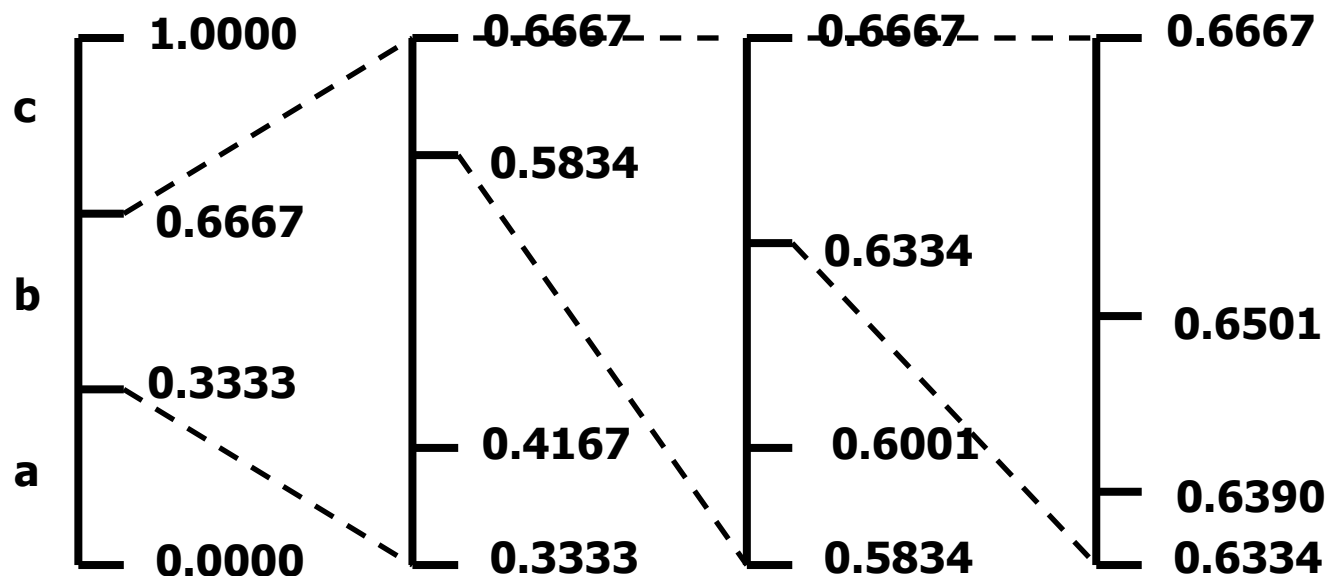
最后用[0.23,0.236)表示eai。

自适应算术编码

输入序列为: bcc.....

假设编码之前我们并不知道信源的概率模型

自适应概率模型:
先假设所有信源符号出现的概率相等, 然后再根据输入符号自适应地调整概率模型



c	1/3	1/4	2/5	3/6
b	1/3	2/4	2/5	2/6
a	1/3	1/4	1/5	1/6

算术编码

- **总结如下特点：**

- 用一个实数表示一个消息串
- 将任意一个消息串映射到 $[0, 1)$ 区间中的一个点
- 当然由于计算机所能表示的实数精度有限，不能实现对无穷消息到一个点的压缩
- 体现了从模糊到精确的表现

游程编码

- 无论是Huffman编码还是算术编码，需要得到先验知识——信源的字符频率，然后进行压缩。但是在大多数情况下，这种先验知识是很难预先获得。因此，设计一种更为通用的数据压缩编码显得尤为重要。
- 游程编码和词典编码**核心思想**：利用数据的**重复结构信息**来进行数据压缩。

游程编码

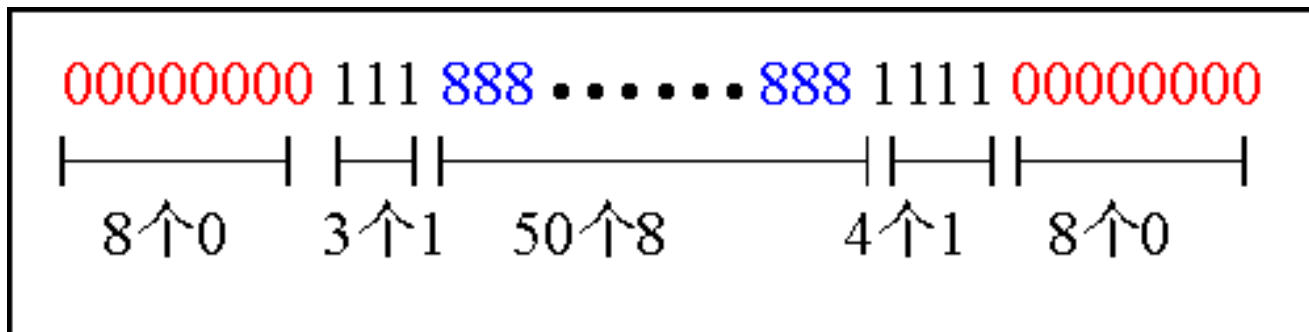
- 现实中有许多这样的图像，在一幅图像中具有许多颜色相同的图块。在这些图块中，许多行上都具有相同的颜色，或者在一行上有许多连续的像素都具有相同的颜色值
- 游程编码（Run-Length Encoding）：它通过将信源中相同符号序列转换成一个计数字段再加上一个重复字符标志实现压缩

游程编码

- 用RLE编码方法得到的代码为：

8 0 3 1 50 8 4 1 8 0

- 代码中用粗体表示的数字是游程长度，其后的数字代表像素的颜色值。例如**50**代表有连续50个像素具有相同的颜色值，它的颜色值是8
- 压缩比约为：7 : 1

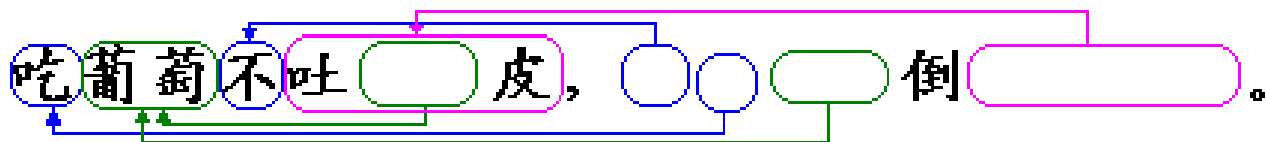


游程编码

- RLE所能获得的压缩比有多大，这主要是取决于图像本身的特点
 - 如果图像中具有相同颜色的图像块越大，图像块数目越少，获得的压缩比就越高。反之，压缩比就越小
 - RLE通常需要和其他的压缩编码技术联合应用

词典编码

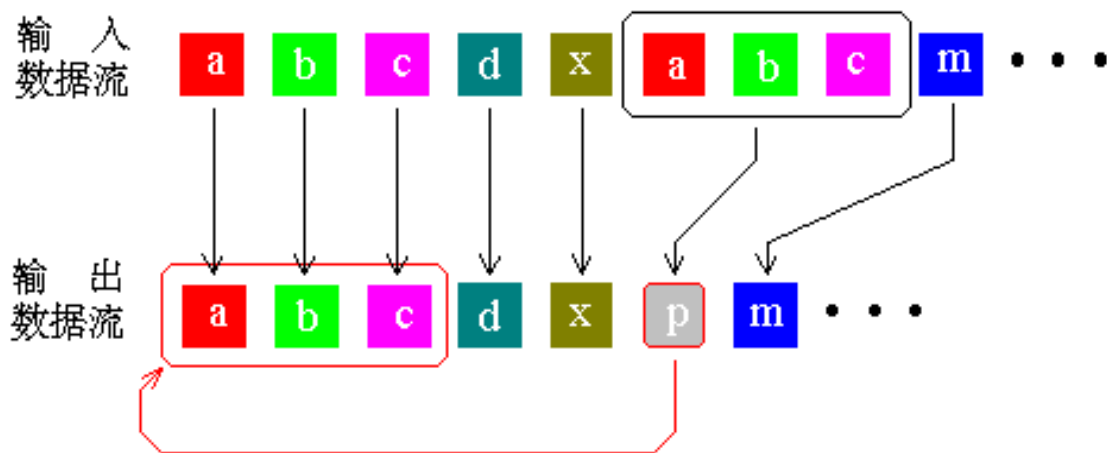
- 词典编码主要利用数据本身包含许多重复代码的特性（文本文件和光栅图像）
 - 例如：吃葡萄不吐葡萄皮，不吃葡萄倒吐葡萄皮。我们用一些简单的代号代替这些字符串，就可以实现压缩，实际上就是利用了信源符号之间的相关性。字符串与代号的对应表就是词典



- 实用的词典编码算法的核心就是如何动态地形成词典，以及如何选择输出格式以减小冗余

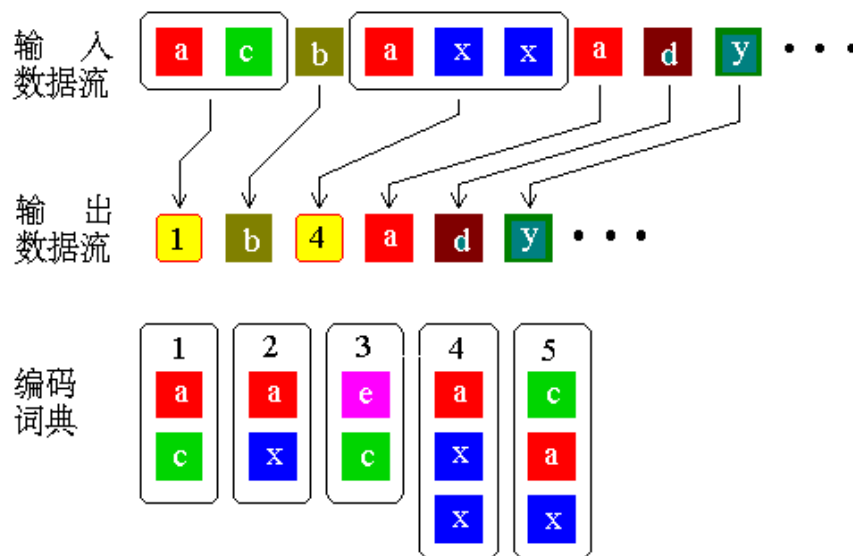
词典编码

- 第一类词典法编码的思想：
 - 企图查找正在压缩的字符序列是否在以前输入的数据中出现过，然后用已经出现过的字符串替代重复的部分，它的输出仅仅是指向早期出现过的字符串的“指针”



词典编码

- 第二类词典法编码的思想：
 - 企图从输入的数据中创建一个“短语词典 (dictionary of the phrases)”，这种短语可以是任意字符的组合。编码数据过程中当遇到已经在词典中出现的“短语”时，编码器就输出这个词典中的短语的“索引号”，而不是短语本身



词典编码

- 第一类词典法
 - LZ77算法：1977年，A. Lempel和J. Ziv开发的算法
 - LZSS算法：1982年，由Storer和Szymanski改进的算法
- 第二类词典法
 - LZ78算法：1978年，J. Ziv和A. Lempel首次发表了介绍这种编码方法的文章
 - LZW算法：1984年，Terry A. Welch发表了改进这种编码算法的文章

LZ77算法

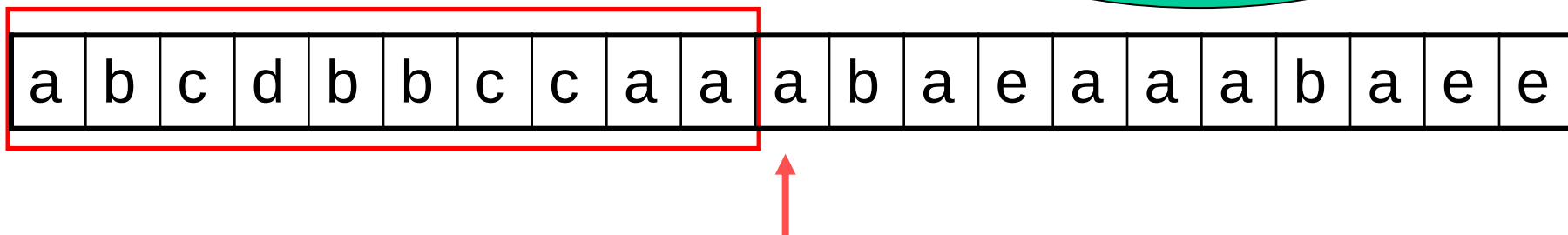
- 1、从当前压缩位置开始，考察未编码的数据，并试图在滑动窗口中找出最长的匹配字符串，如果找到，则进行步骤 2，否则进行步骤 3
- 2、输出三元符号组 (off, len, c)。其中 off 为窗口中匹配字符串相对窗口边界的偏移，len 为可匹配的长度，c 为下一个字符。然后将窗口向后滑动 len + 1 个字符，继续步骤 1
- 3、输出三元符号组 (0, 0, c)。其中 c 为下一个字符。然后将窗口向后滑动 len + 1 个字符，继续步骤 1



LZ77算法

- 假设窗口的大小为 10 个字符，我们刚编码过的 10 个字符是：abcbbbcaca，即将编码的字符串头 abaeaaabae

窗口和编码位置
向右移动 $2 + 1$

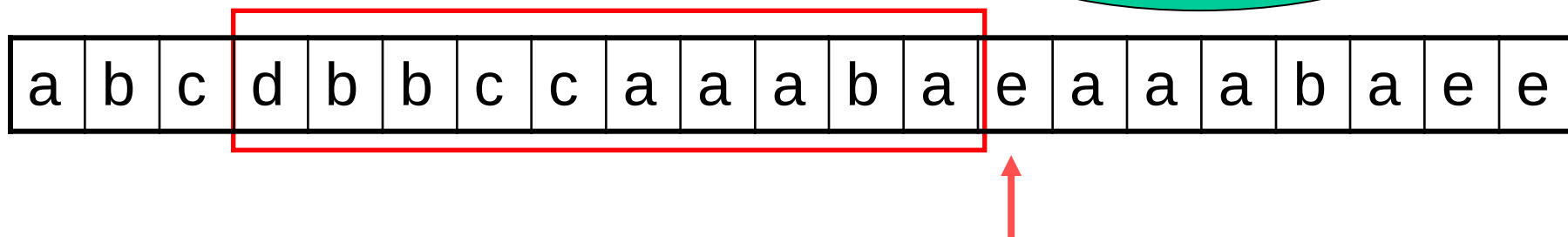


步骤	位置	匹配串	输出
1	1	ab	(0,2) a
2	4	--	(0,0) e
3	5	aaabae	(4,6) e

LZ77算法

- 假设窗口的大小为 10 个字符，我们刚编码过的 10 个字符是：abcbbbcaca，即将编码的字符串头 abaeaaabaee

窗口和编码位置
向右移动 $0 + 1$

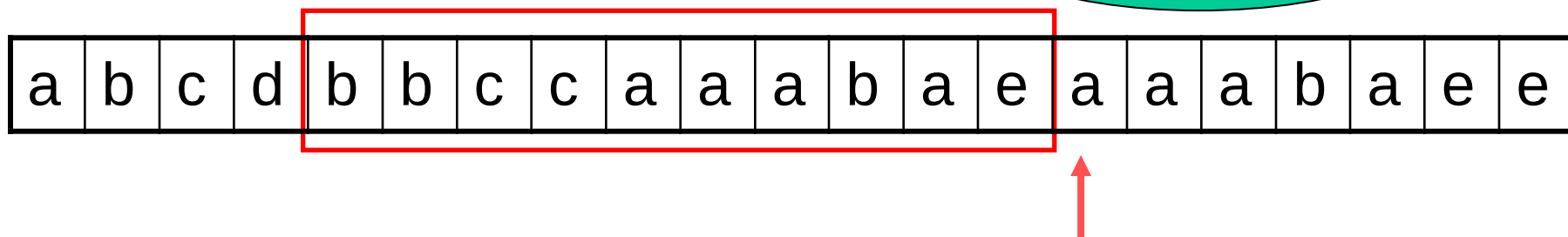


步骤	位置	匹配串	输出
1	1	ab	(0,2) a
2	4	--	(0,0) e
3	5	aaabae	(4,6) e

LZ77算法

- 假设窗口的大小为 10 个字符，我们刚编码过的 10 个字符是：abcbbbcaca，即将编码的字符串头 abaeaaabaee

窗口和编码位置
向右移动 6 + 1



步骤	位置	匹配串	输出
1	1	ab	(0,2) a
2	4	--	(0,0) e
3	5	aaabae	(4,6) e

LZ77算法

- 解压缩的过程十分简单，只要我们像压缩时那样维护好滑动的窗口，随着三元组的不断输入，我们在窗口中找到相应的匹配串，缀上后继字符 c 输出（如果 off 和 len 都为 0 则只输出后继字符 c ）即可还原出原始数据。

LZSS算法

- LZ77通过输出真实字符解决了在窗口中出现没有匹配串的问题，但这个解决方案包含有冗余信息
 - 冗余信息表现在两个方面，一是空指针，二是编码器可能输出额外的字符，这种字符是指可能包含在下一个匹配串中的字符
- LZSS算法的思想
 - 如果匹配串的长度比指针本身的长度长就输出指针(匹配串长度 $\geq$ MIN_LENGTH)，否则就输出真实字符。另外要输出额外的标志位区分是指针还是字符

LZSS算法

- LZSS编码算法的具体执行步骤如下：
 - 1、把编码位置置于输入数据流的开始位置。
 - 2、在前向缓冲存储器中查找与窗口中最长的匹配串
 - ① Pointer : =匹配串指针。
 - ② Length : =匹配串长度。
 - 3、判断匹配串长度Length是否大于等于最小匹配串长度($\text{Length} \geq \text{MIN_LENGTH}$)
 - 如果“是”：输出指针，然后把编码位置向前移动Length个字符
 - 如果“否”：输出前向缓冲存储器中的第1个字符，然后把编码位置向前移动一个字符。
 - 4、如果前向缓冲存储器不是空的，就返回到步骤 2

LZSS算法

输入数据流:

位置	1	2	3	4	5	6	7	8	9	10	11
字符	A	A	B	B	C	B	B	A	A	B	C

编码过程
MIN_LEN
=2

步骤	位置	匹配串	输出
1	1	--	A
2	2	A	A
3	3	--	B
4	4	B	B
5	5	--	C
6	6	BB	(3, 2)
7	8	AAB	(7, 3)
8	11	C	C

LZSS算法

- 在相同的计算机环境下，LZSS算法比LZ77可获得比较高的压缩比，而译码同样简单
 - 许多后来开发的文档压缩程序都使用了LZSS的思想。例如，PKZip, GZip, ARJ, LHArc和ZOO等等，其差别仅仅是指针的长短和窗口的大小等有所不同
- LZSS同样可以和熵编码联合使用
 - 例如ARJ就与霍夫曼编码联用，而PKZip则与Shannon-Fano联用，它的后续版本也采用霍夫曼编码

预测编码

预测编码

- 基本思想：
 - 根据某一模型利用以往的样本值对于新的样本值进行预测，然后将样本的实际值与其预测值相减得到一个误差值，对这一误差值进行编码
 - 如果模型足够好且样本序列在时（空）间上的相关性较强，误差信号的幅度将远远小于原始信号，因此可使用较少的位数对其量化

DPCM的基本原理

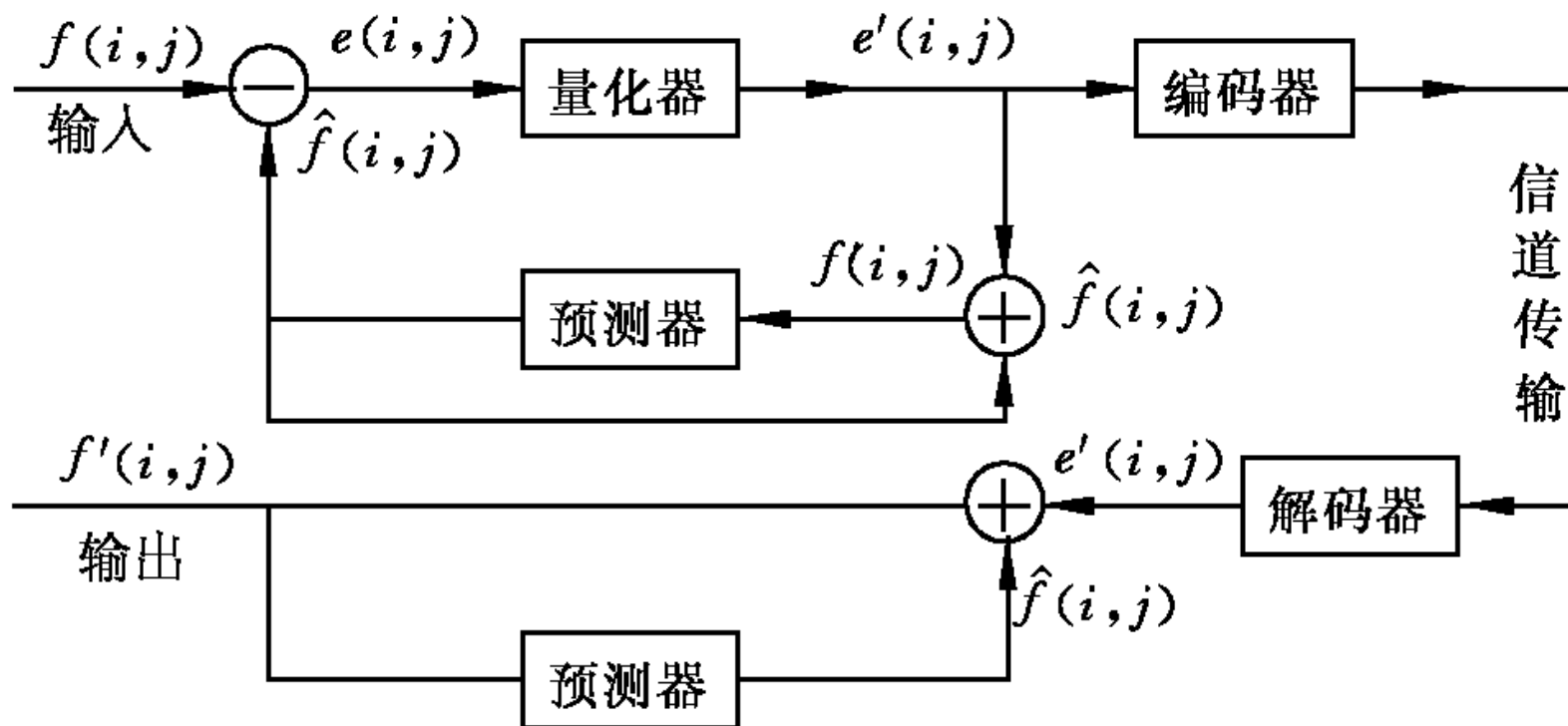
- 一幅二维静止图像，像素点 (i, j) 的灰度实际灰度是 $f(i, j)$ ， $f'(i, j)$ 是根据以前已出现的像素点的灰度对该点的预测灰度，也称为**预测值或估计值**。计算预测值得像素，可以是同一行扫描的前几个像素，或者前几行上的像素，甚至是前几帧的邻近像素。
- 实际值和预测值之间的差值：

$$e(i, j) = f(i, j) - \hat{f}(i, j)$$

DPCM的基本原理

- 由于图像像素之间有极强的相关性，所以这个预测误差是很小的。编码时，不是对像素点的实际灰度 $f'(i, j)$ 进行编码，而是对预测误差 $e(i, j)$ 进行量化、编码、发送。

DPCM的基本原理



预测误差:
$$e(i, j) = f(i, j) - \hat{f}(i, j)$$

DPCM的基本原理

- 由像素(i, j)的3个相邻像素得到的三点预测，定义为：

$$\hat{f}(i, j) = a_1 f(i, j-1) + a_2 f(i-1, j-1) + a_3 f(i-1, j)$$

构成三阶预测器。其中 a_1, a_2, a_3 称预测系数，都是待定参数。预测误差为：

$$\begin{aligned} e(i, j) &= f(i, j) - \hat{f}(i, j) \\ &= f(i, j) - [a_1 f(i, j-1) \\ &\quad + a_2 f(i-1, j-1) \\ &\quad + a_3 f(i-1, j)] \end{aligned}$$

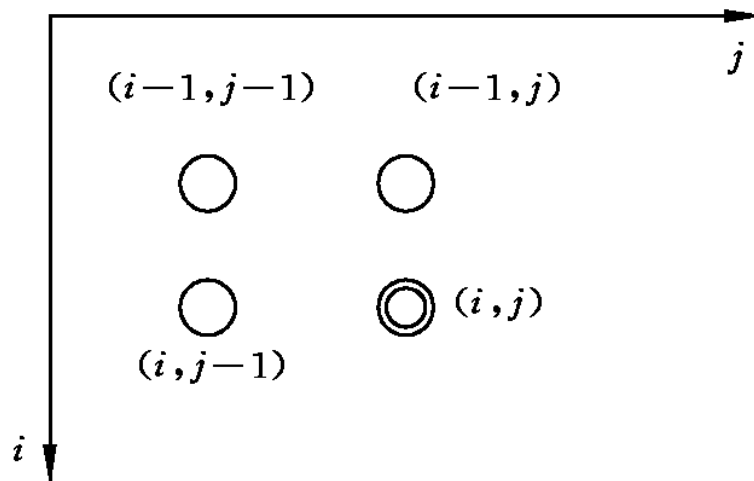
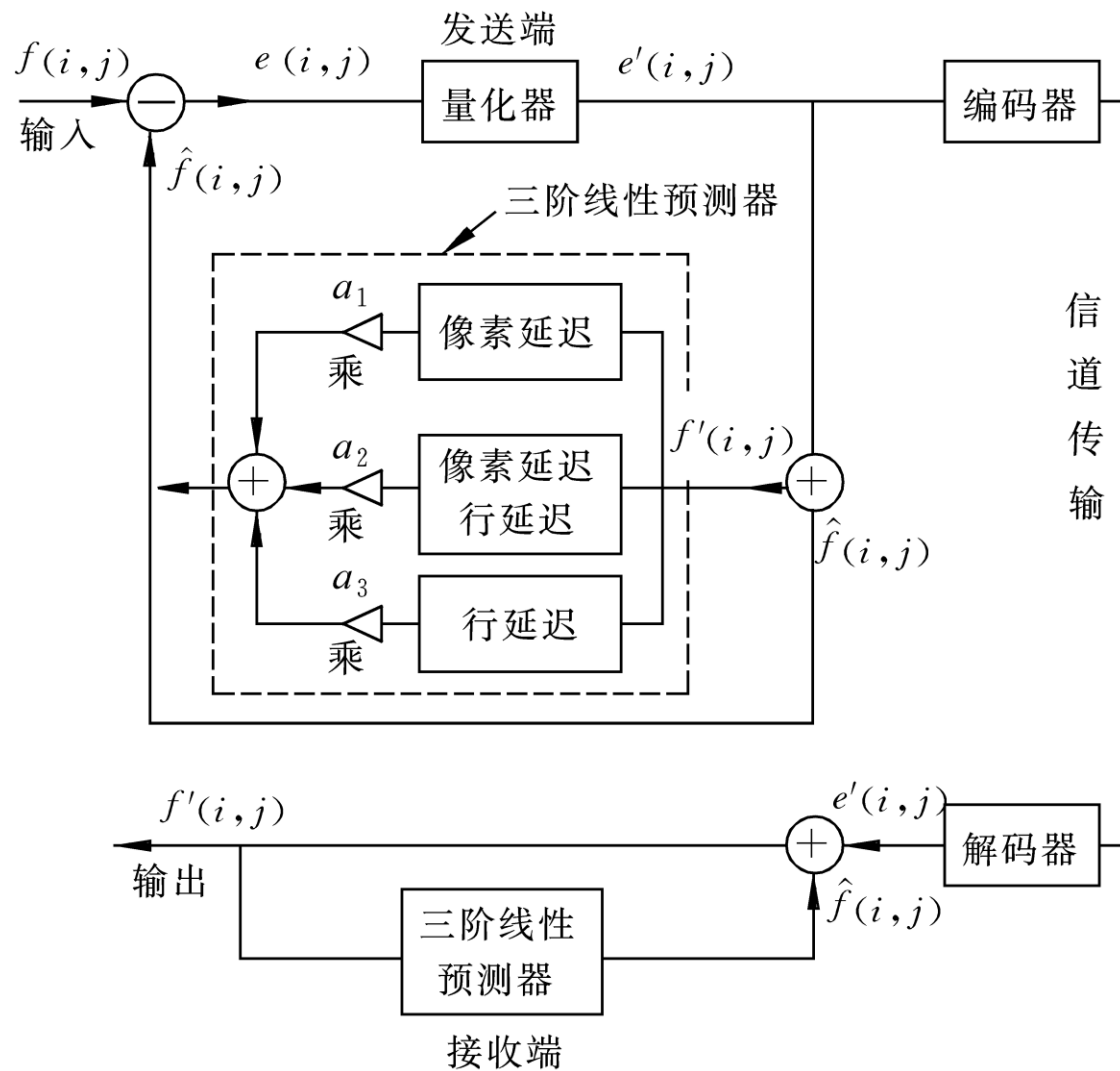


图 4.14 预测域

DPCM的基本原理

三阶DPCM 线性预测



DPCM的基本原理

- **基于均方误差最小准则的最佳线性预测**

- 均方误差的表达式为：

$$\begin{aligned}\bar{e}^2 &= E\{e(i, j)^2\} = E\{[f(i, j) - \hat{f}(i, j)]^2\} \\ &= E\{[f(i, j) - a_1 f(i, j-1) - a_2 f(i-1, j-1) - a_3 f(i-1, j)]^2\}\end{aligned}$$

- 将预测值与实际值之间的均方误差对 a_1, a_2, a_3 求偏导, 令

$$\frac{\partial \bar{e}^2}{\partial a_1} = 0, \quad \frac{\partial \bar{e}^2}{\partial a_2} = 0, \quad \frac{\partial \bar{e}^2}{\partial a_3} = 0,$$

- 解方程, 得 a_1, a_2, a_3 , 即为最佳线性预测系数

自适应预测编码

- **DPCM系统**

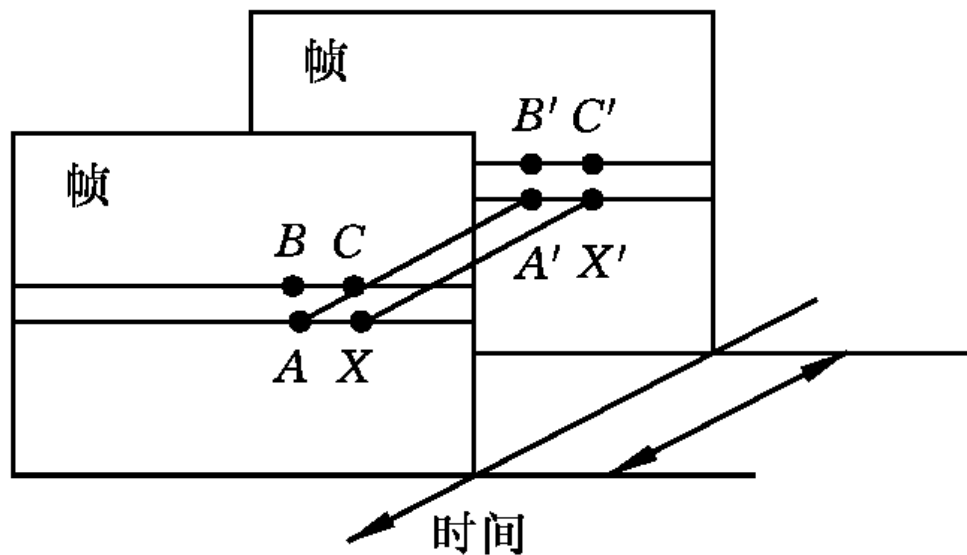
- 当预测系数和量化器参数一次设计好后，整幅图都用这套参数，不再改变

- **自适应差分脉冲编码调制（ADPCM）系统**

- 自适应预测和自适应量化：预测器的预测系数和量化器的量化参数，能够根据图像的局部区域分布特点而自动调整
- 实践证明：ADPCM与DPCM相比，不仅能改善恢复图像的评测质量和视觉效果，同时还能进一步压缩数据

帧间预测编码

- 帧间编码技术处理的对象是序列图像（运动图像）
- 基于预测技术的帧间预测编码方法：
 - (1) 条件补充法
 - (2) 运动补偿技术



条件补充法

- **条件像素补充法**

- 若帧间各对应像素的亮度差超过阈值，则把这些像素存入缓冲存储器，并以恒定的传输速度传送；而阈值以下的像素则不传送，在接收端用上一帧相应像素值代替
- 一幅电视图像可能只传送其中较少部分的像素，且传送的只是帧间差值，压缩比较高
- 可视电话中，只需传送6%左右的像素

运动补偿技术

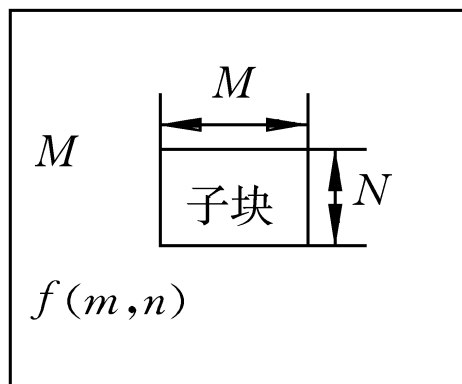
- **运动补偿技术**

- 是MPEG标准中使用的主要技术之一。尤其适合于运动部分只占整个画面较小的电视会议、可视电话，引入运动补偿后，压缩比可以提高很多。
- 运动补偿方法是跟踪画面内的运动情况，对其加以补偿之后再进行帧间预测，关键是运动向量的计算。

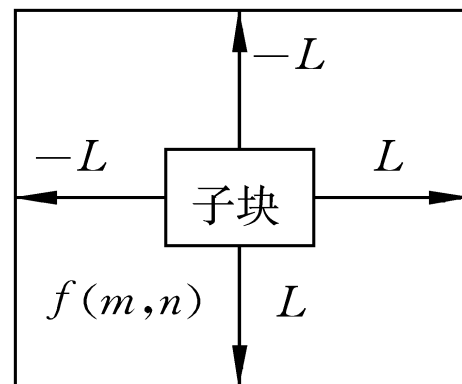
运动补偿技术

- **运动补偿技术**

- 把图像分为若干字块图像，设子图像是 $M \times N$ 的矩形块。当前帧图像亮度信号为 $f_k(m, n)$ ，前一次传送的图像为 $f_{k-N_s}(m, n)$ ， N_s 是帧差数目，通常为1、3或7。当前帧的一个子块是从第 $k - N_s$ 帧平移而来。假设运动物体在 N_s 帧差时间内水平和垂直最大位移均为 L ，可以在第 $k - N_s$ 帧搜索区SR内搜索，SR搜索区大小为 $(M+2L, N+2L)$



第 K 帧（当前帧）



第 $K - N_s$ 帧

运动补偿技术

- 块匹配算法

- 1、判别两个子块匹配的准则

$$NCCF(i, j) = \frac{\sum_{m=1}^M \sum_{n=1}^N f_k(m, n) f_{k-N_s}(m+i, n+j)}{[\sum_{m=1}^M \sum_{n=1}^N f_k^2(m, n)]^{1/2} [\sum_{m=1}^M \sum_{n=1}^N f_{k-N_s}^2(m+i, n+j)]^{1/2}}$$

$$MSE(i, j) = \sum_{m=1}^M \sum_{n=1}^N [f_k(m, n) - f_{k-N_s}(m+i, n+j)]^2, \quad (i, j) \in SR$$

$$MAD(i, j) = \sum_{m=1}^M \sum_{n=1}^N |f_k(m, n) - f_{k-N_s}(m+i, n+j)|, \quad (i, j) \in SR$$

- 2、计算量最小的搜索方法

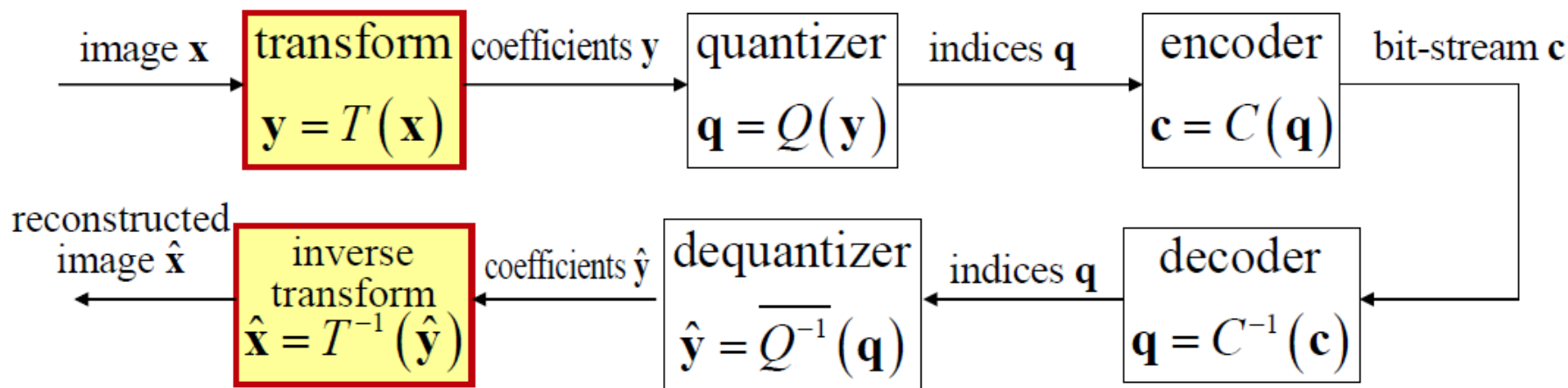
- 全搜索、二维对数法、三步法、共轭方向法、正交搜索法等

变换编码

变换编码基本思想

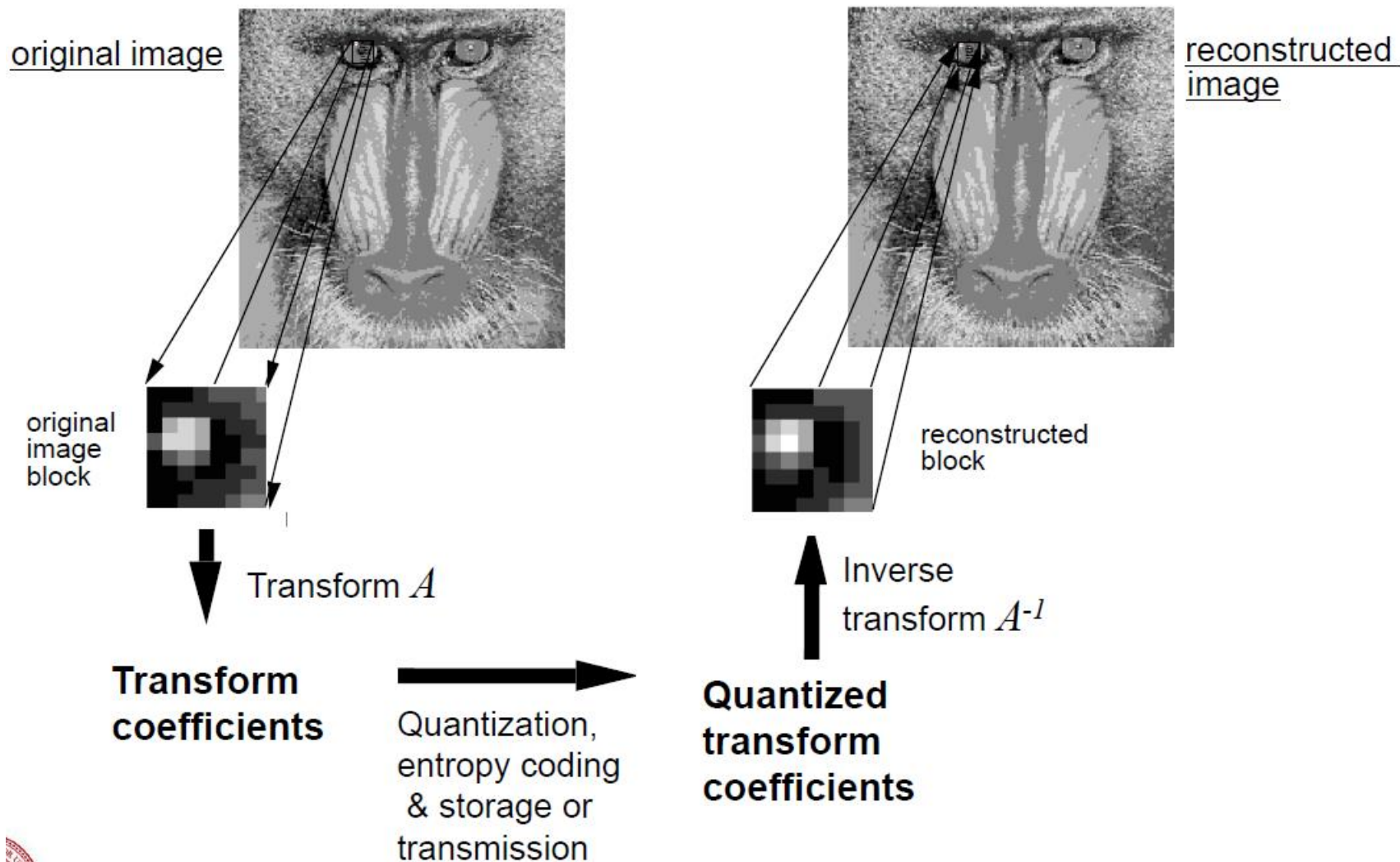
- 变换编码不是直接对空域图像信号编码，而是首先将空域图像信号映射变换到另一个正交矢量空间（变换域或频域），产生一批变换系数，然后对这些变换系数进行编码处理
- 正交变换种类
 - 傅里叶(Fourier)变换
 - K-L (Karhunen-Loeve)变换
 - 余弦变换、正弦变换
 - 沃尔什(Walsh)变换
 - 哈尔(Haar)变换等

变换编码框图



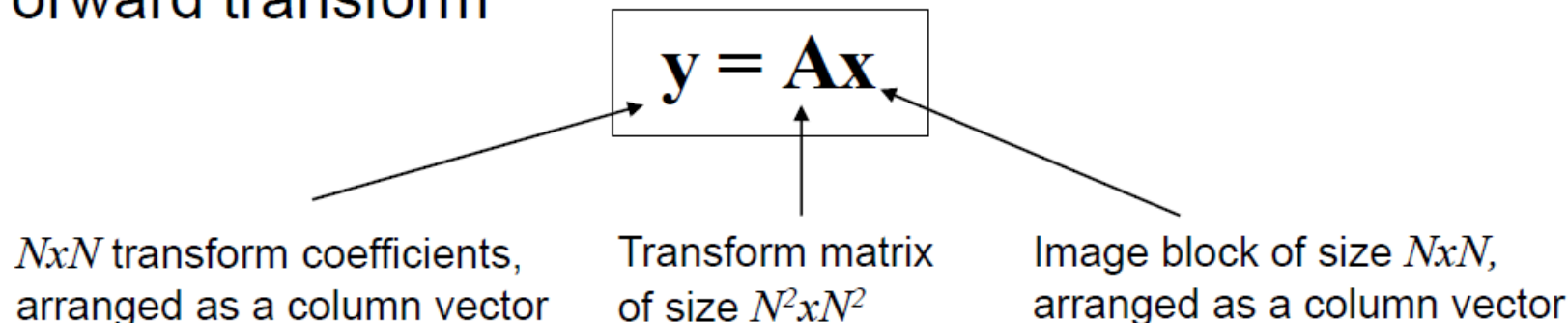
- Transform $T(\mathbf{x})$ usually invertible
- Quantization $Q(\mathbf{y})$ not invertible, introduces distortion
- Combination of encoder $C(\mathbf{q})$ and decoder $C^{-1}(\mathbf{c})$ lossless

基于块的变换编码



正交变换性质

- Forward transform



- Inverse transform

$$\mathbf{x} = \mathbf{A}^{-1} \mathbf{y} = \mathbf{A}^T \mathbf{y}$$

- Linearity: $\mathbf{x}$ is represented as linear combination of “basis functions” (i.e., columns of $\mathbf{A}^T$)

能量守恒

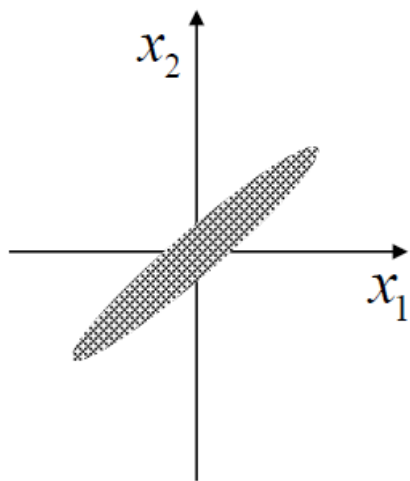
- For any orthonormal transform $\mathbf{y} = \mathbf{A}\mathbf{x}$

$$\|\mathbf{y}\|^2 = \mathbf{y}^T \mathbf{y} = \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x} = \|\mathbf{x}\|^2$$

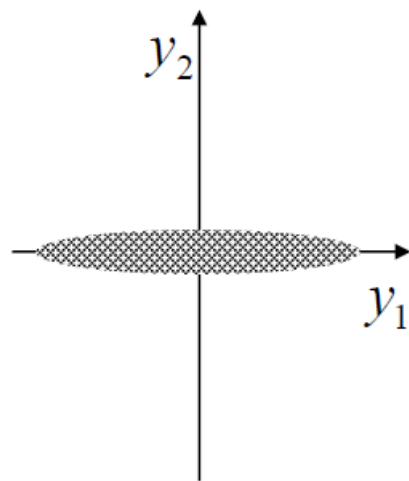
- Interpretation
 - Vector length („energies“) conserved
 - Orthonormal transform is a rotation of the coordinate system around the origin (plus possible sign flips)

二维正交变换

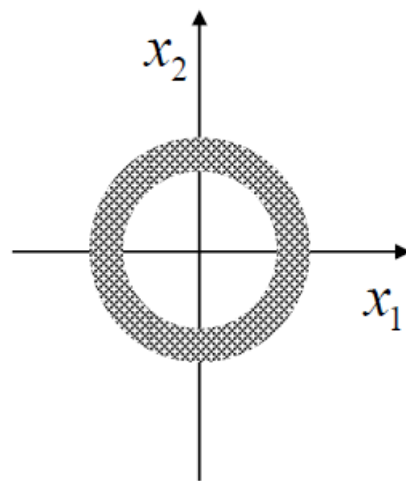
$$\mathbf{A} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$



Strongly correlated
samples,
equal energies

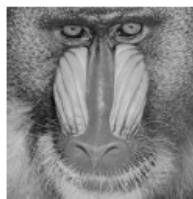
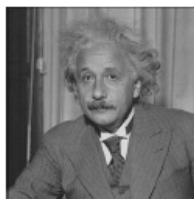
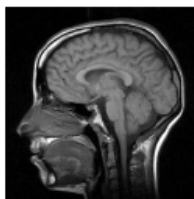
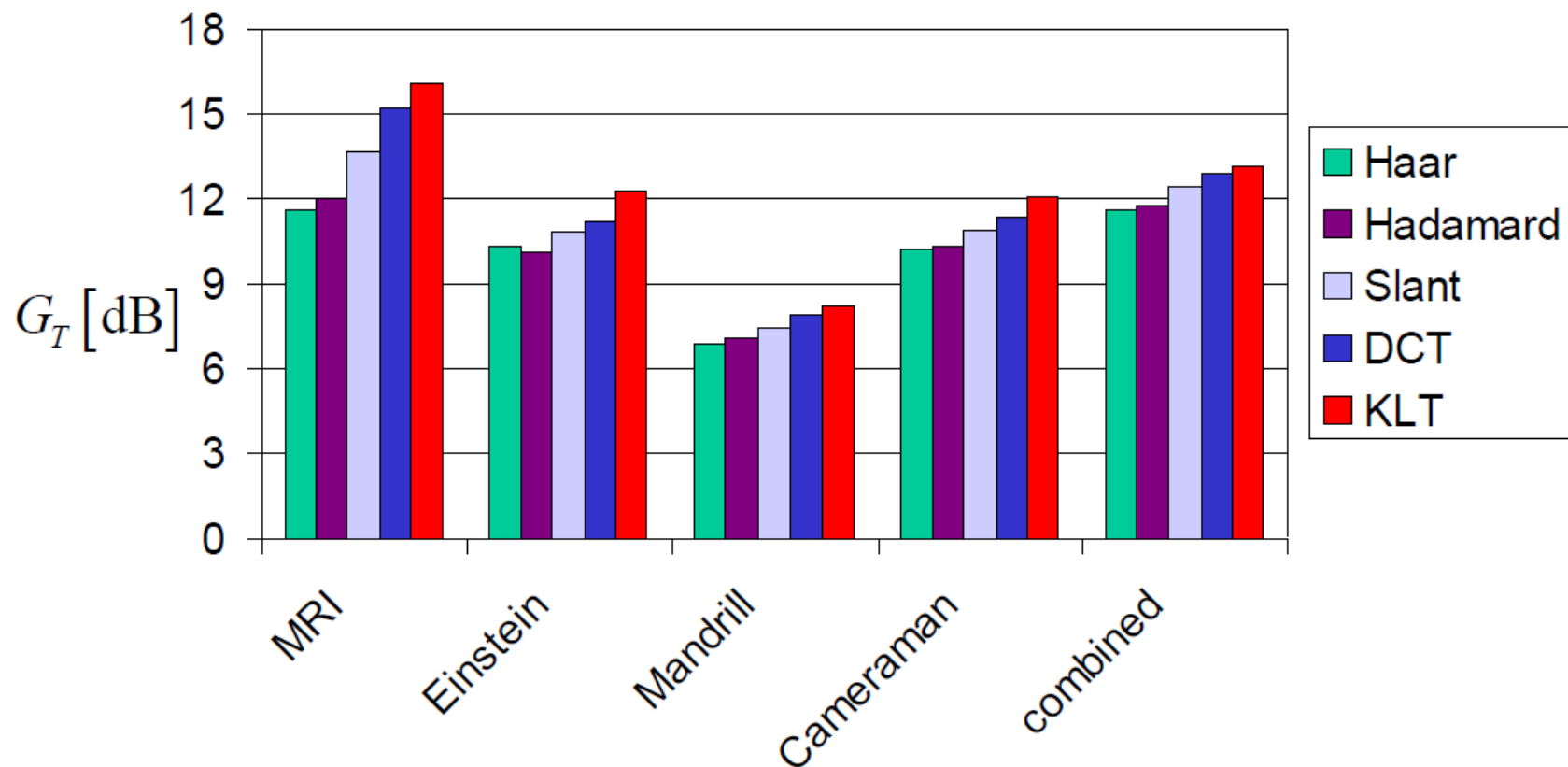


After transform:
uncorrelated samples,
most of the energy in
first coefficient



Despite statistical
dependence, orthonormal
transform won't help.

编码增益



离散余弦变换 (DCT)

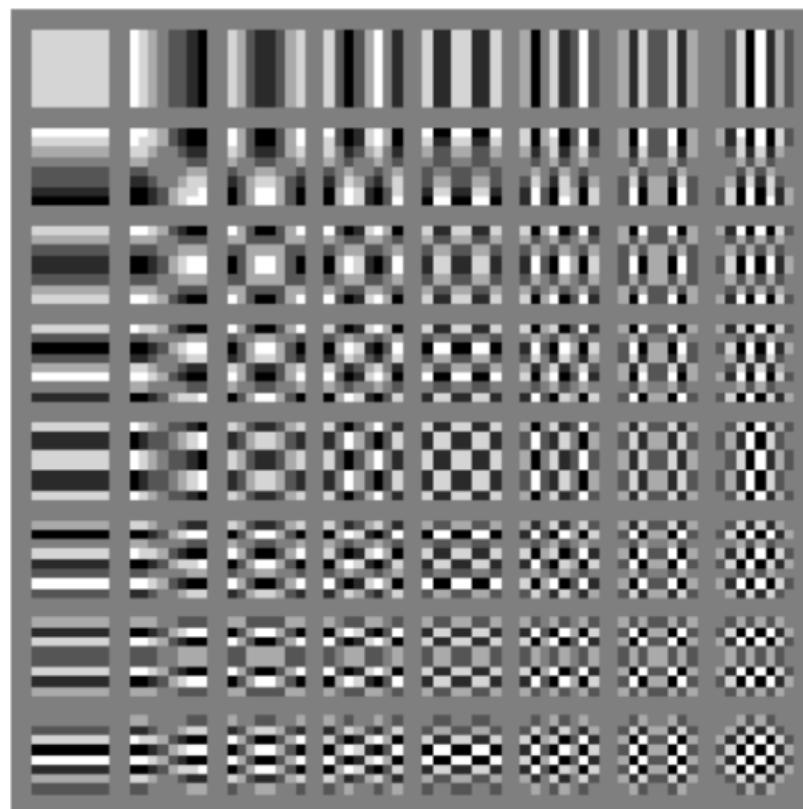
- Type II-DCT of blocksize $N \times N$ is defined by transform matrix A containing elements
- 2D DCT basis functions:

$$a_{ik} = \alpha_i \cos \frac{\pi(2k+1)i}{2N}$$

for $i, k = 0, \dots, N-1$

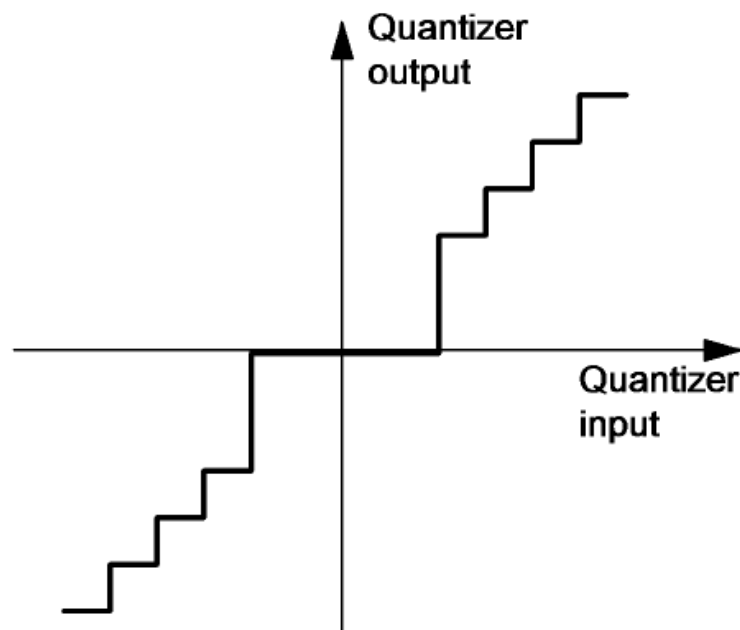
$$\text{with } \alpha_0 = \sqrt{\frac{1}{N}}$$

$$\alpha_i = \sqrt{\frac{2}{N}} \quad \forall i \neq 0$$



对变换系数的编码

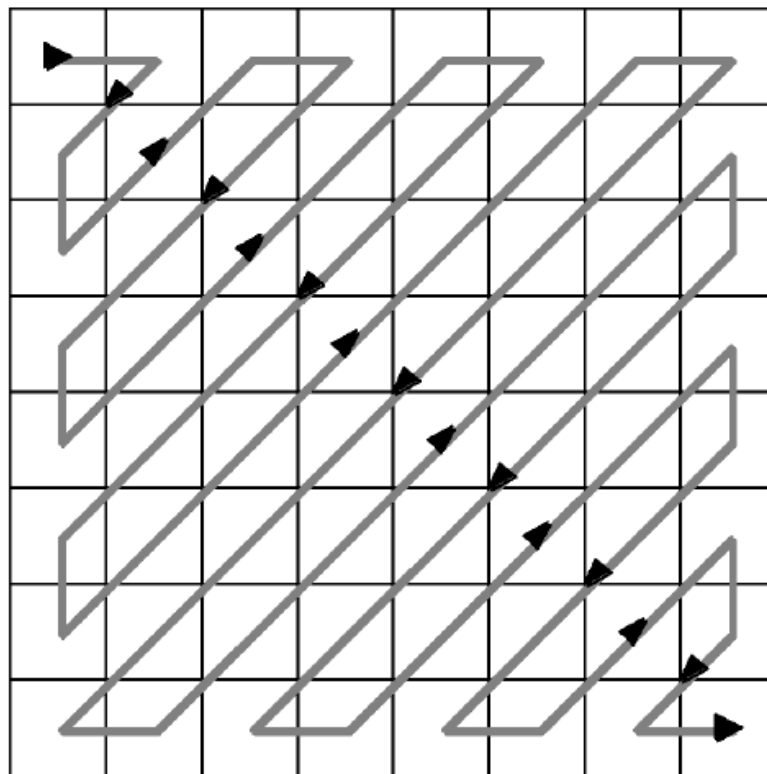
- Uniform deadzone quantizer: transform coefficients that fall below a threshold are discarded.



- Positions of non-zero transform coefficients are transmitted in addition to their amplitude values.

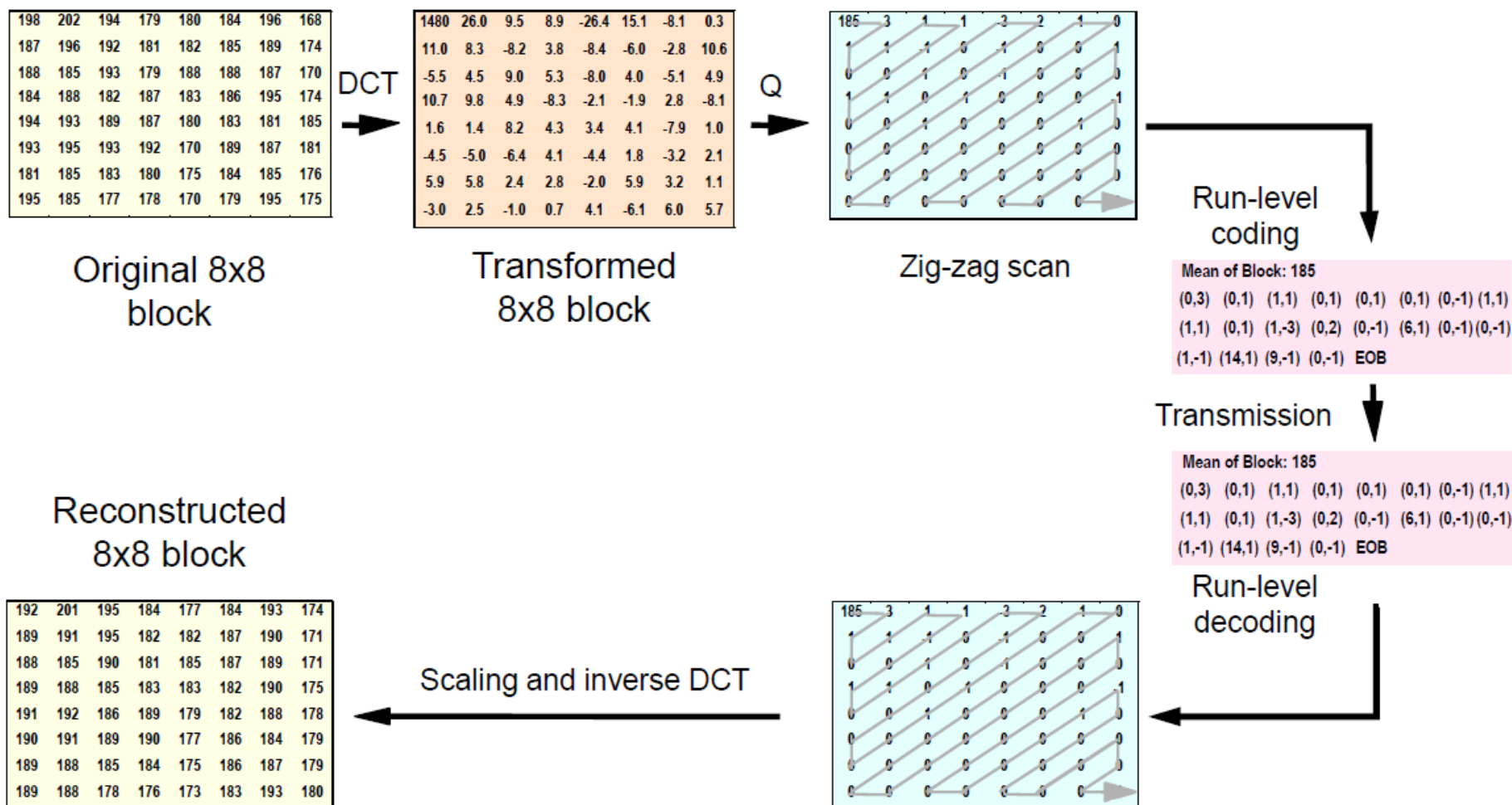
对变换系数的编码

- Efficient encoding of the position of non-zero transform coefficients: zig-zag-scan + run-level-coding



ordering of the transform coefficients by zig-zag-scan

对变换系数的编码



DCT编码失真

DCT coding with increasingly coarse quantization, block size 8x8



quantizer stepsize
for AC coefficients: 25

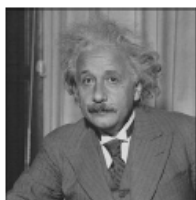
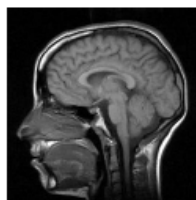
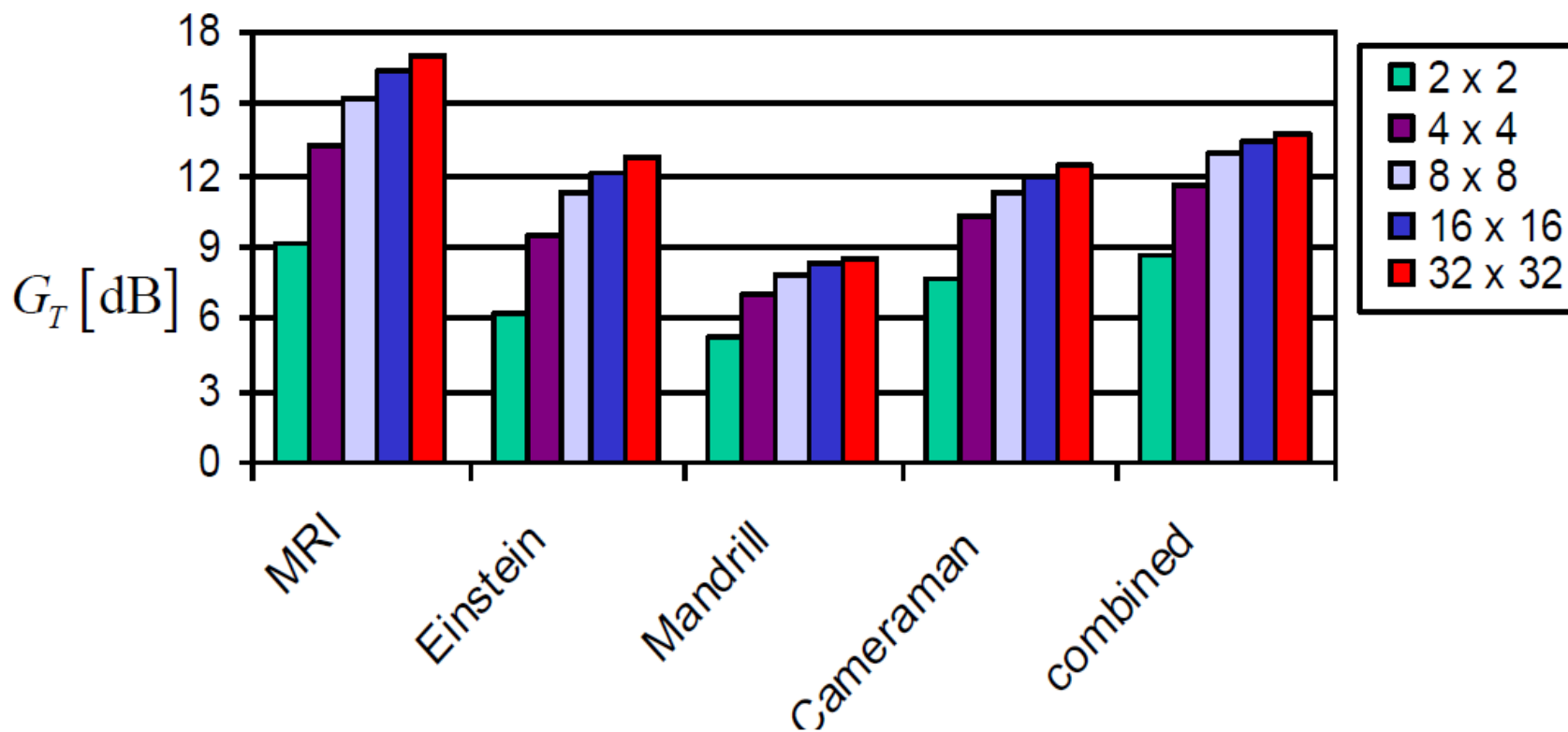


quantizer stepsize
for AC coefficients: 100



quantizer stepsize
for AC coefficients: 200

图像块尺寸的影响



变换编码小结

- Orthonormal transform: rotation of coordinate system in signal space
- Purpose of transform: decorrelation, energy concentration
- Bit allocation proportional to logarithm of variance, equal distortion
- KLT is optimum, but signal dependent and, hence, without a fast algorithm
- DCT shows reduced blocking artifacts compared to DFT
- 8x8 block size, uniform quantization, zig-zag-scan + run-level coding is widely used today (e.g. JPEG, MPEG, ITU-T H.261, H.263)